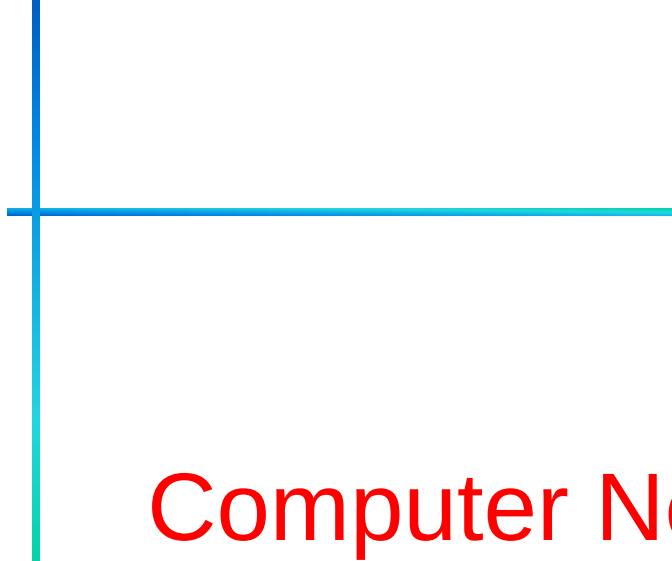
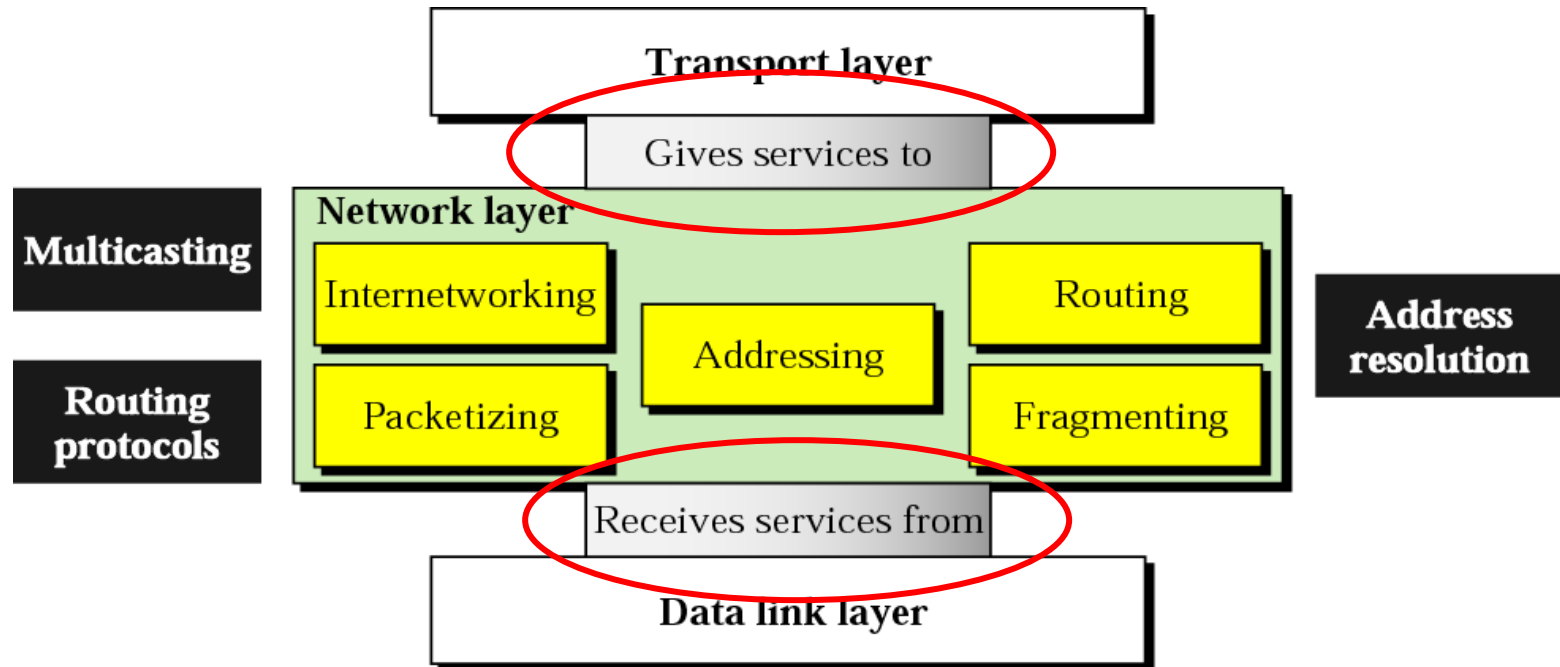




Computer Networks

Network Layer

Position of the network layer

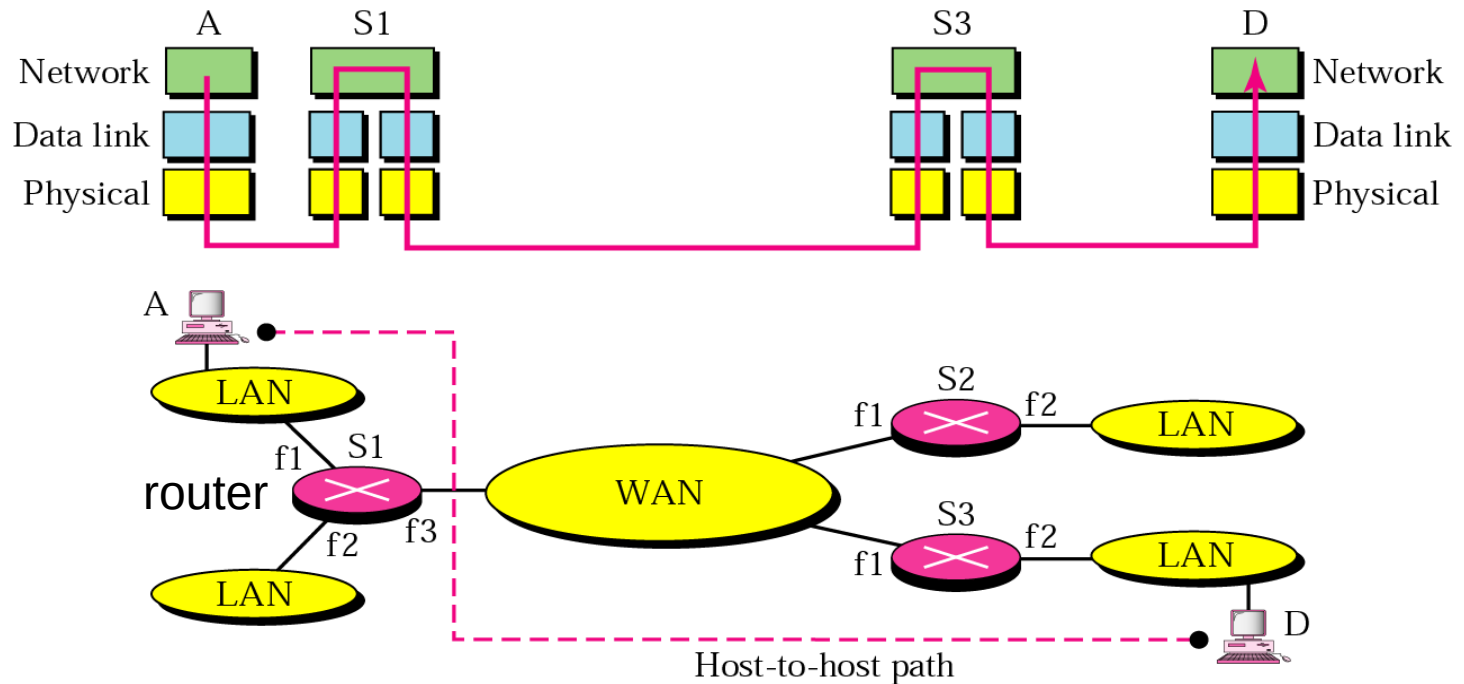


- piece of the network layer in each and every host and router in the network
 - unlike upper layers

Network layer

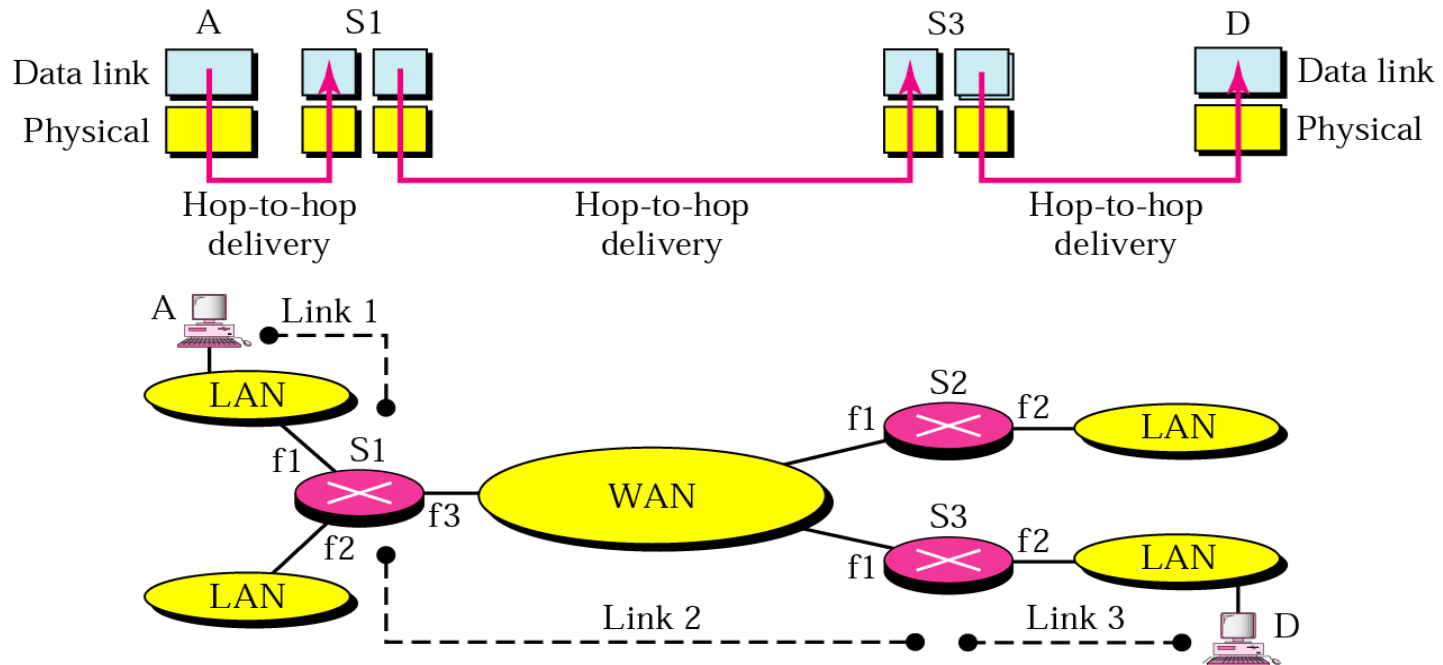
- Responsible for the delivery of individual packets from source host to destination host
- Major duties
 - Internetworking
 - making that all the physical networks look like a single network
 - Addressing
 - *uniquely & universally* define the connection of a node in the network
 - Routing
 - packet transport through the network via different routes
 - Packetizing
 - Transport layer data (segments) encapsulation
 - Fragmenting
 - Breaking an arbitrary size datagrams into smaller pieces
- In some network – connection setup

Internetworking



- Internetwork made out of 4 LANs and 1 WAN
- network-to-network data transmission

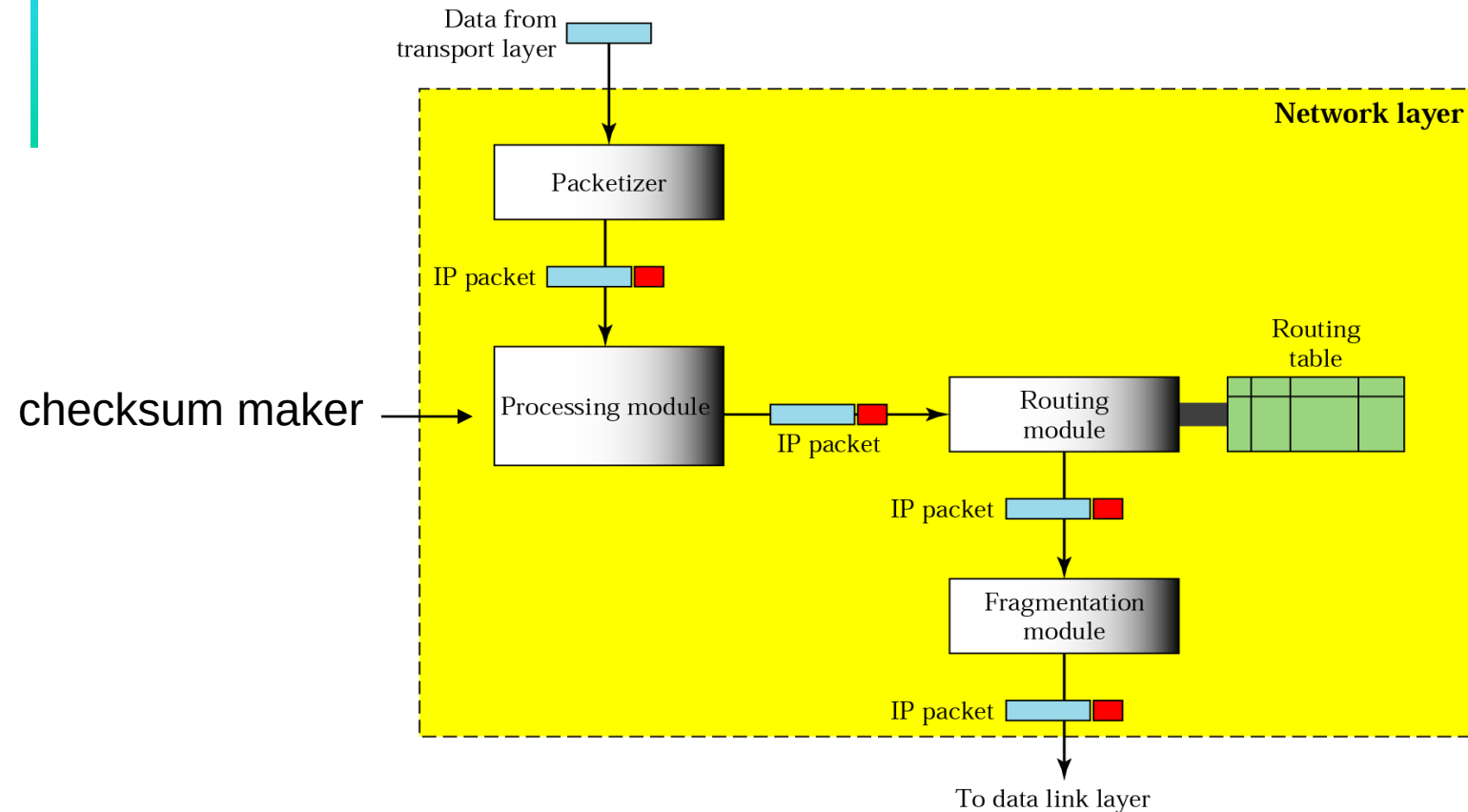
Links in an internetworking



- How does router S1 know that data arrived at f1 have to be sent out on f3?
- Solⁿ, Introduce network layer

General Functionality of Network layer at the source

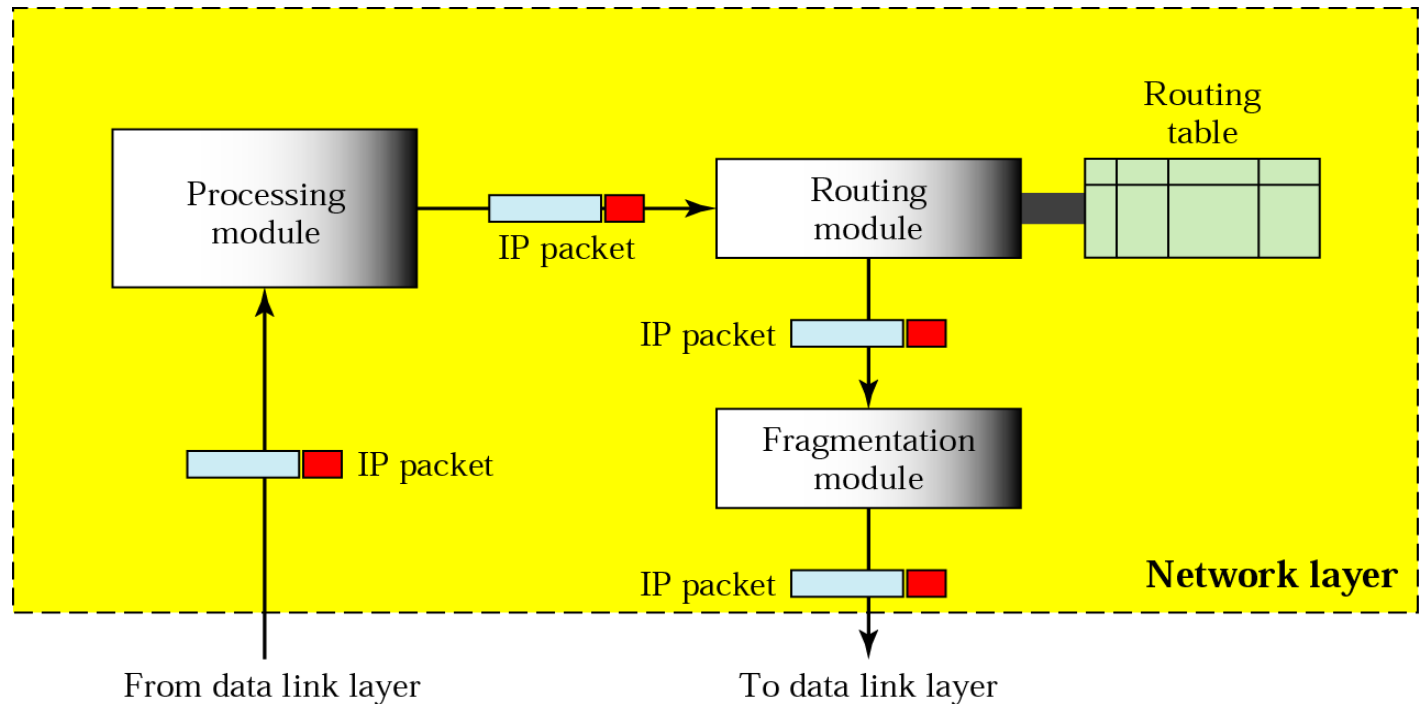
- creates both destination and source address



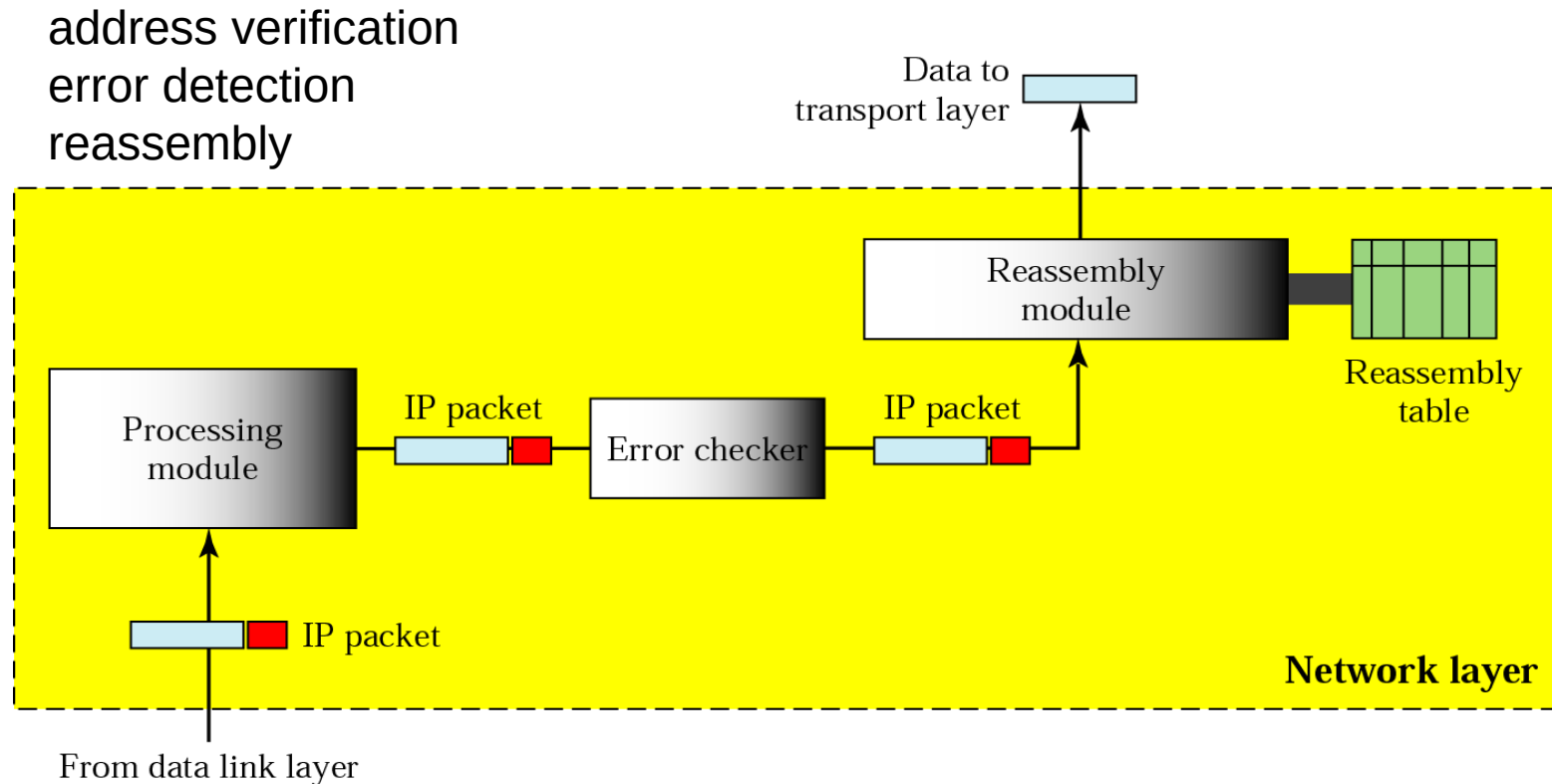
General Functionality of Network layer at the router

fragmentation
optional

no reassembly,
why?

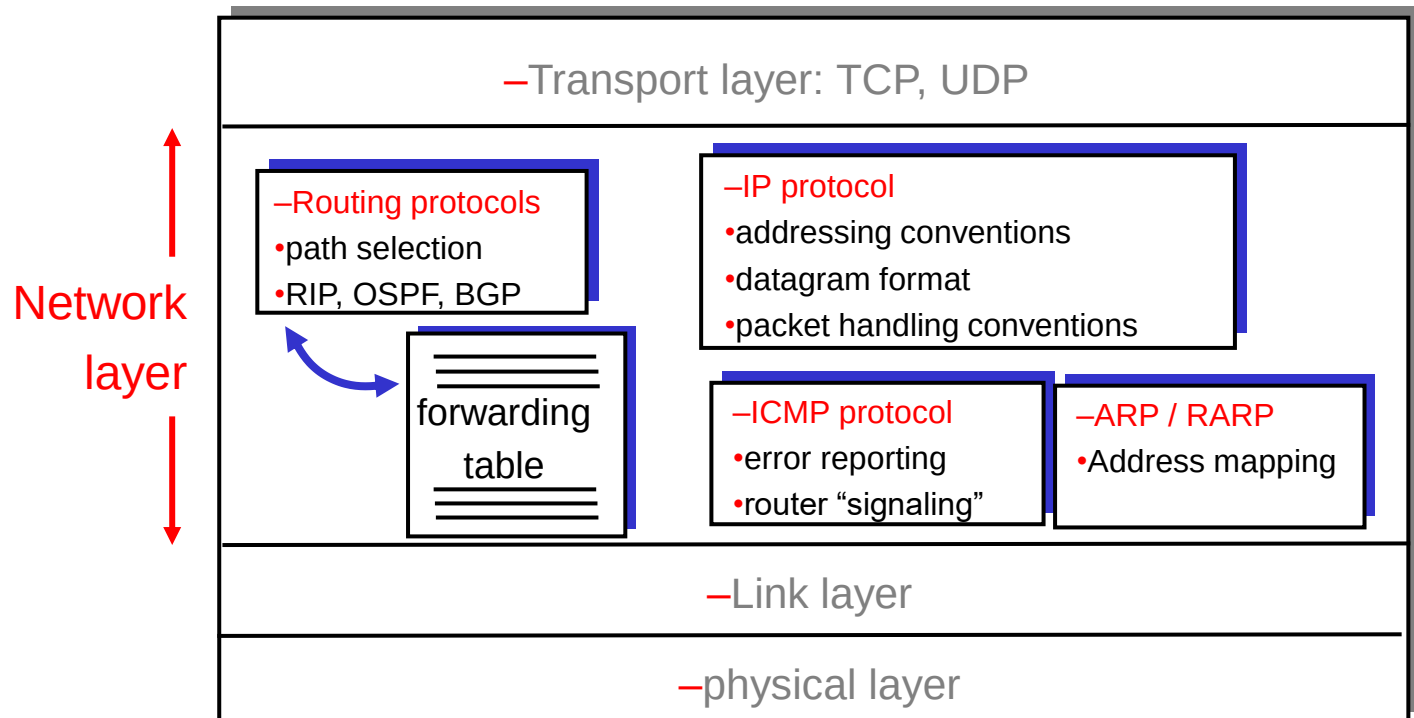


General Functionality of Network layer at the destination



We interested in

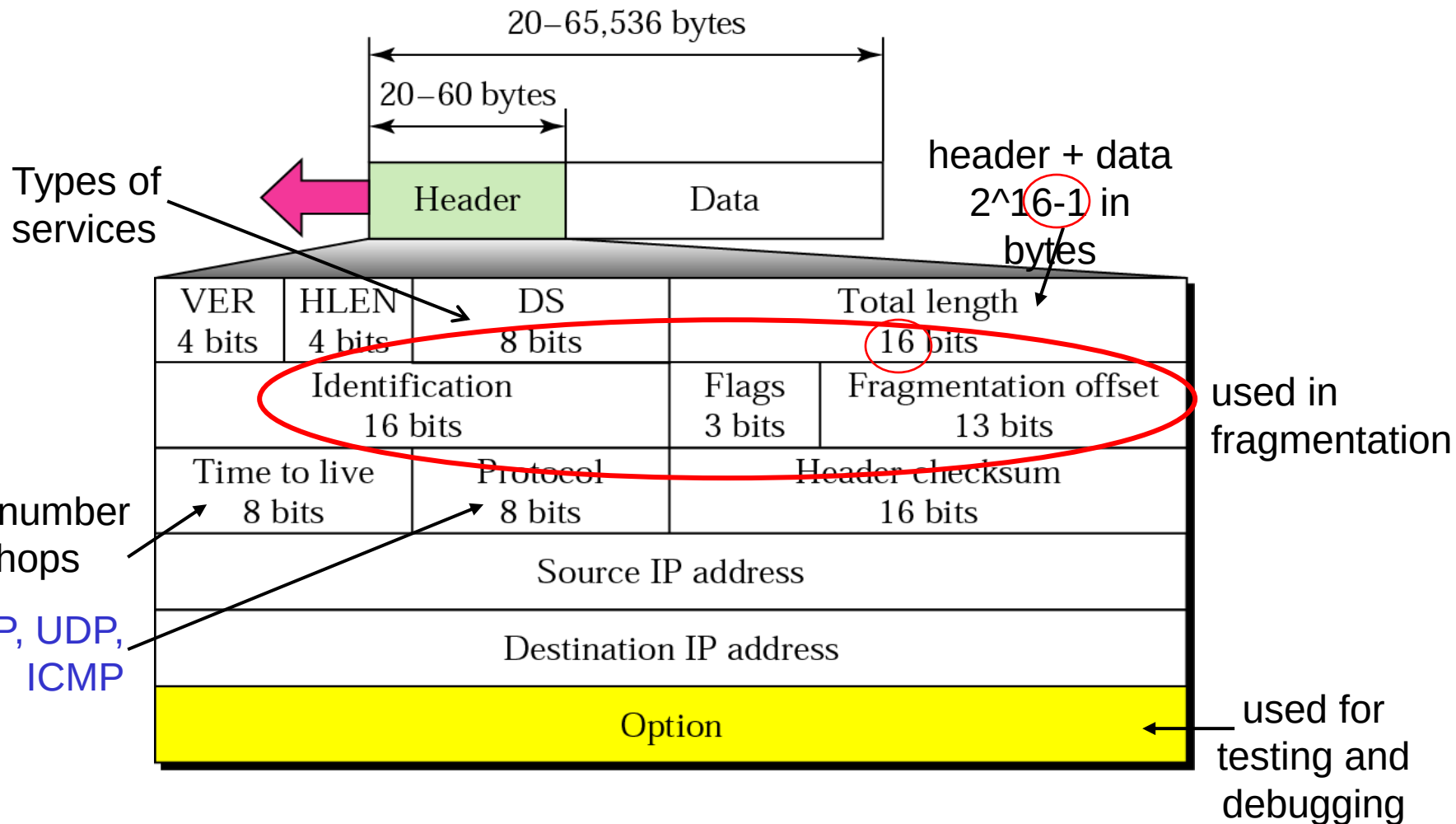
- The network layer of the Internet
- Communication at the network layer in the Internet is connectionless
 - ie no connection setup



Internet Protocol (IP)

- Connection-less unreliable protocol with the best-effort delivery service (why?)
 - Best effort: no error correction or flow control
 - Use error detection: discard the corrupted packet
- Combined with TCP if reliability is important

IPv4 datagram format

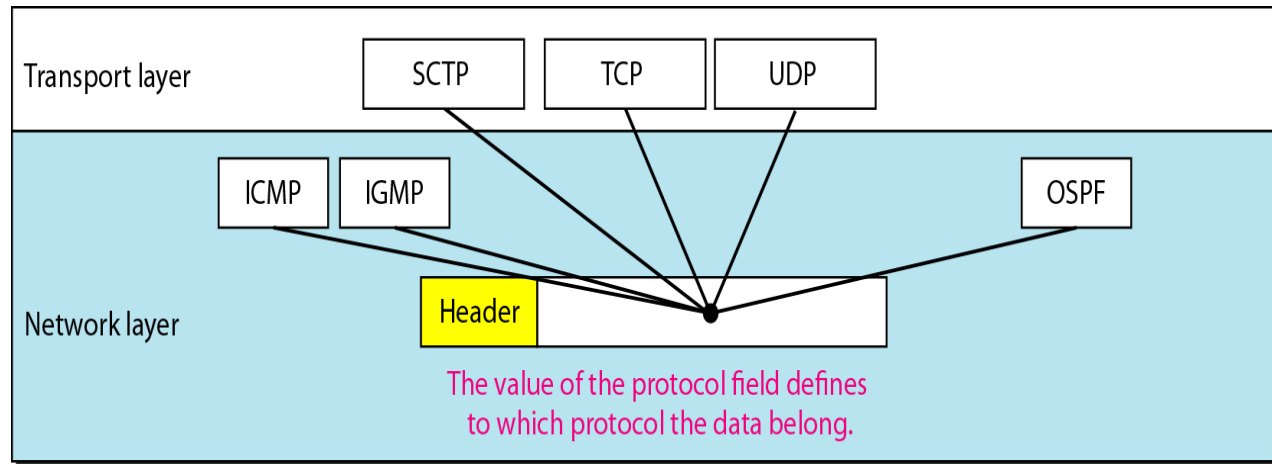


IPv4 datagram format

- **Version (VER)** – 4 bits, specify IP version of the datagram
- **Header Length (HLEN)** – 4 bits, defines total length of header in quad word (4 byte),
 - eg – if no option, header will be 20 bytes and then HLEN = 5
- **Differentiated Services (DS)** – 8 bits, allows different types of IP datagram to be distinguished from each other
 - eg – to distinguish real time datagram (ip telephony) from non real time traffic (FTP)
- **Datagram Length** – 16 bits, total length of datagram in bytes
- **Identification, Flag, Offset** – discussed later
- **Time to Live (TTL)** – 8 bits, used to ensure that datagram will not circulate forever
 - each router decrements this value, if 0, discard the datagram

IPv4 datagram format

- **Protocol** – 8 bits, used in destination host, indicates the specific destination protocol to which the IP datagram is delivered

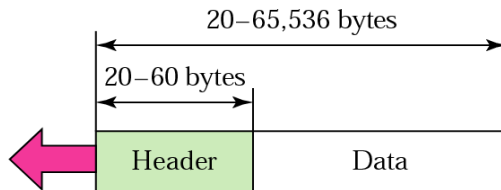


<i>Value</i>	<i>Protocol</i>
1	ICMP
2	IGMP
6	TCP
17	UDP
89	OSPF

IPv4 datagram format

- **Header Checksum** – 16 bits, aids detecting bit error in IP datagram

- Covers only header
- Not data – **Why?**
- eg- calculation



VER 4 bits	HLEN 4 bits	DS 8 bits	Total length 16 bits	
Identification 16 bits			Flags 3 bits	Fragmentation offset 13 bits
Time to live 8 bits		Protocol 8 bits	Header checksum 16 bits	
Source IP address				
Destination IP address				
Option				

4	5	0	28	
1			0	0
4	17	0		
10.12.14.5				
12.6.7.9				

4, 5, and 0 → 0100010100000000
 28 → 00000000000011100
 1 → 00000000000000001
 0 and 0 → 00000000000000000
 4 and 17 → 0000010000010001
 0 → 00000000000000000
 10.12 → 0000101000001100
 14.5 → 0000111000000101
 12.6 → 0000110000000110
 7.9 → 0000011100001001

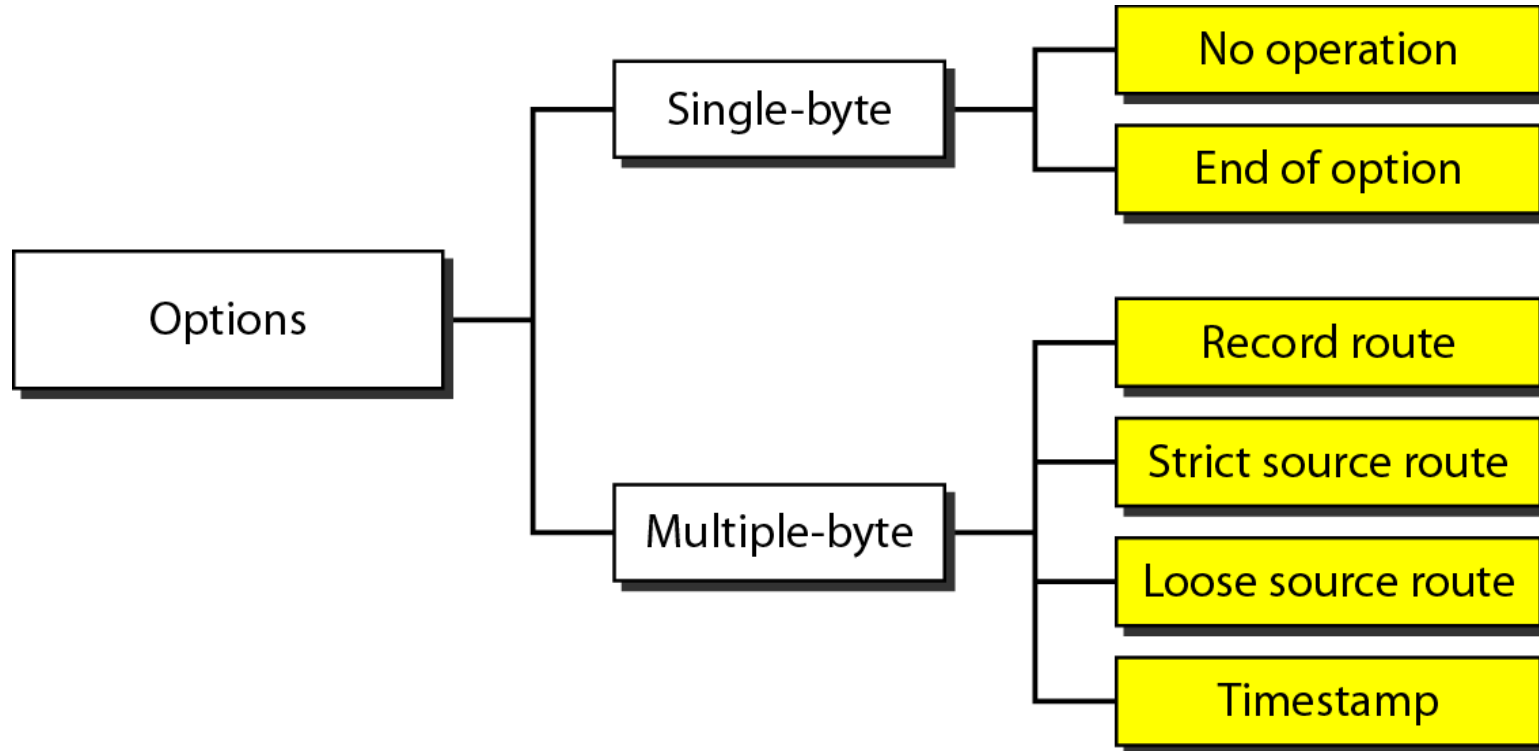
Sum → 0111010001001110
 Checksum → 1000101110110001

IPv4 datagram format

- **Source Address** – 32 bit value defines the IP address of sending host
- **Destination Address** – 32 bit value defines the IP address of the receiving host

IPv4 datagram format

- **Options** – can vary up to 40bytes, used for network testing, debugging

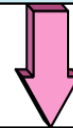
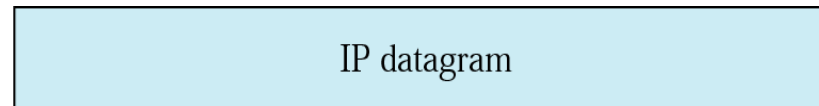


IP Fragmentation & Reassembly

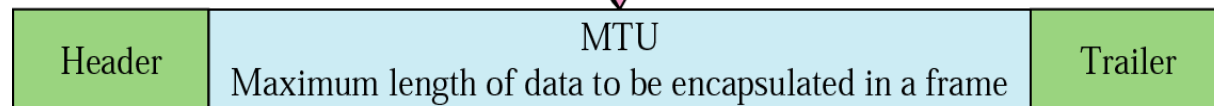
- Each network links have MTU (max. transfer unit) - largest possible link-level frame.
 - different link types, different MTUs

<i>Protocol</i>	<i>MTU</i>
Hyperchannel	65,535
Token Ring (16 Mbps)	17,914
Token Ring (4 Mbps)	4,464
FDDI	4,352
Ethernet	1,500
X.25	576
PPP	296

Network
Layer



Data Link
Layer



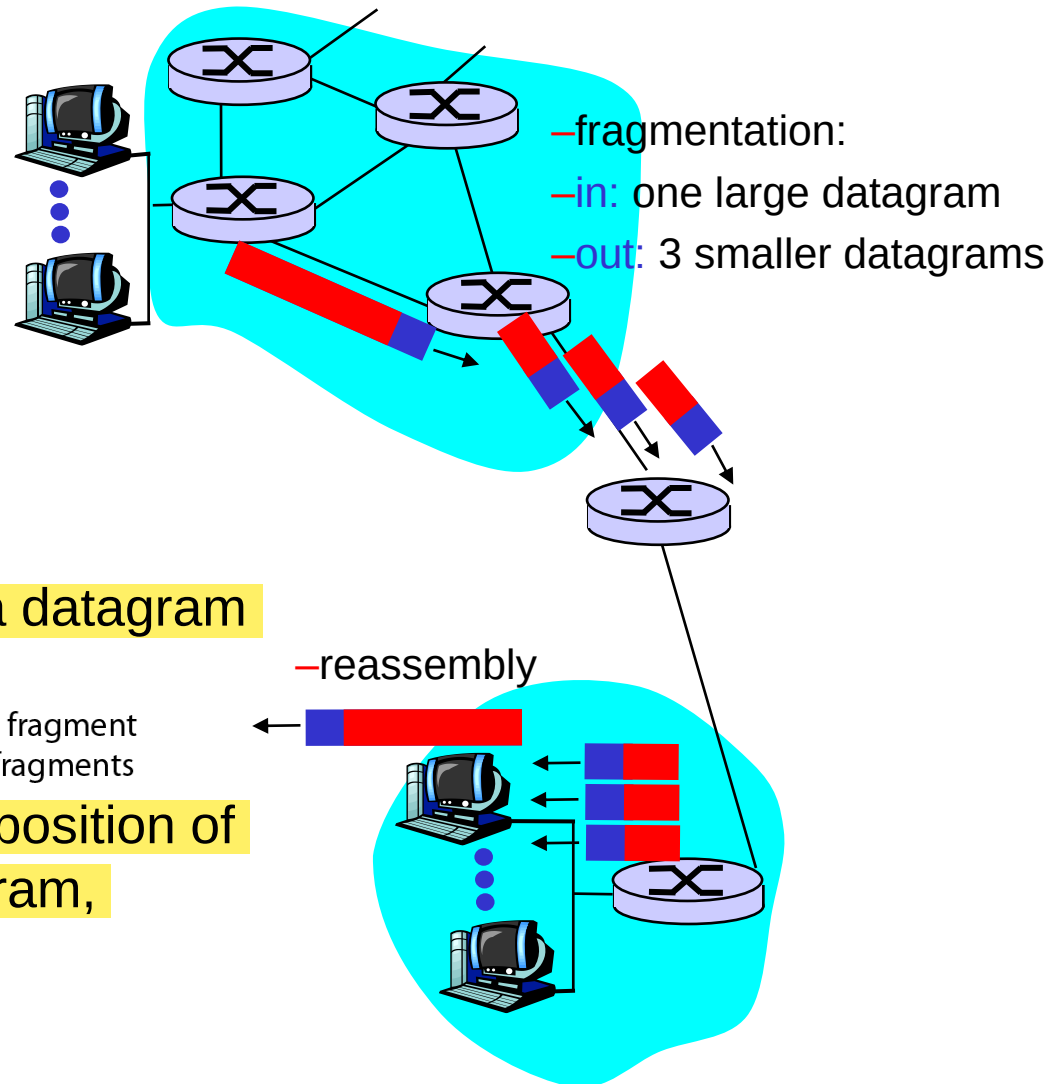
Frame

IP Fragmentation & Reassembly

- large IP datagram divided (“fragmented”) within net
 - one datagram becomes several datagrams
- reassembled only at final destination
- IP header bits used to identify, and order related fragments
- **Identification** - 16 bit, identify a datagram originating from the source host
- **Flag** – 3 bits

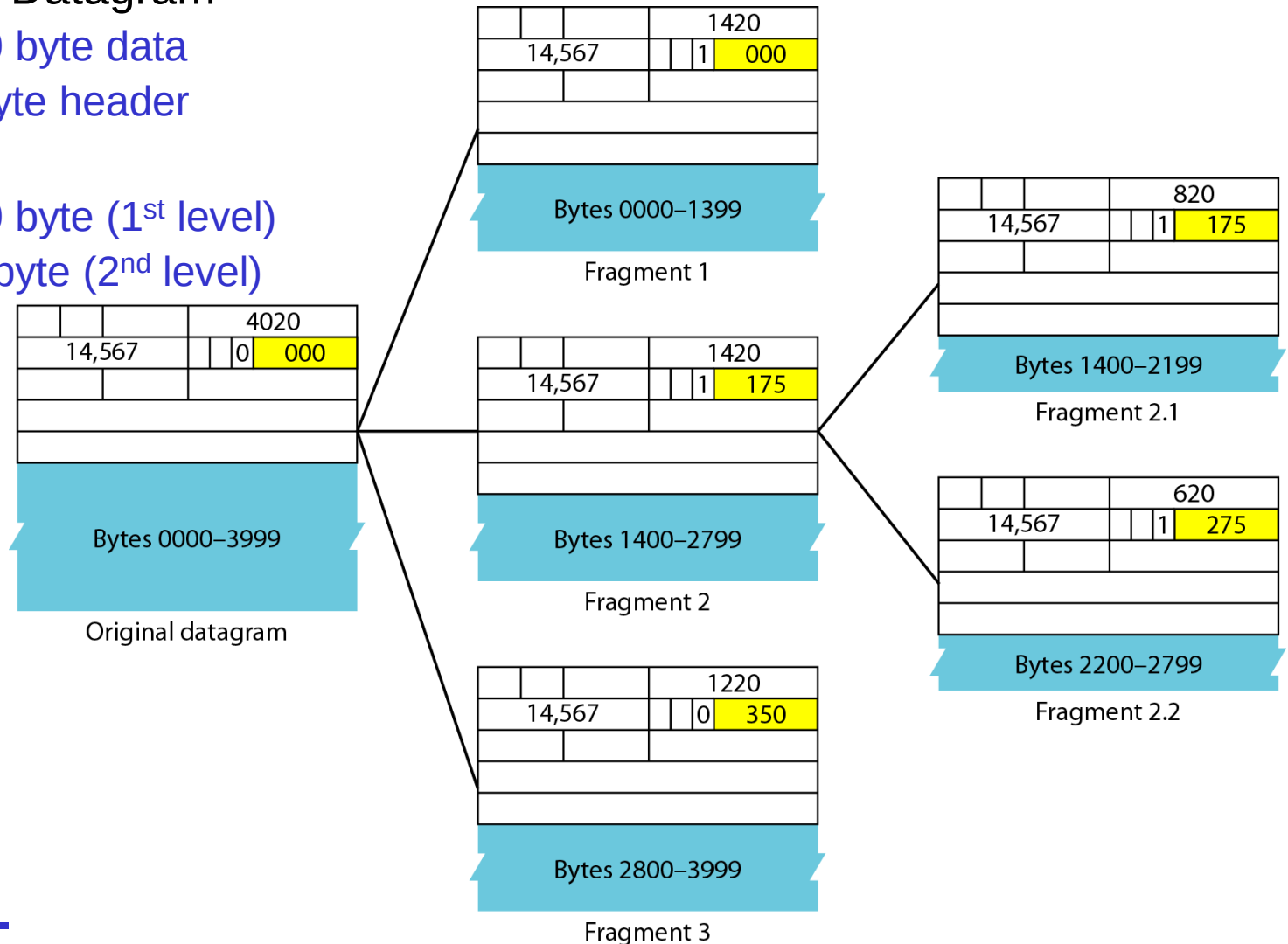
	D	M
--	---	---

D: Do not fragment
M: More fragments
- **Offset** – 13 bits, shows relative position of the fragment to the whole datagram,
 - Measured in 8 bytes

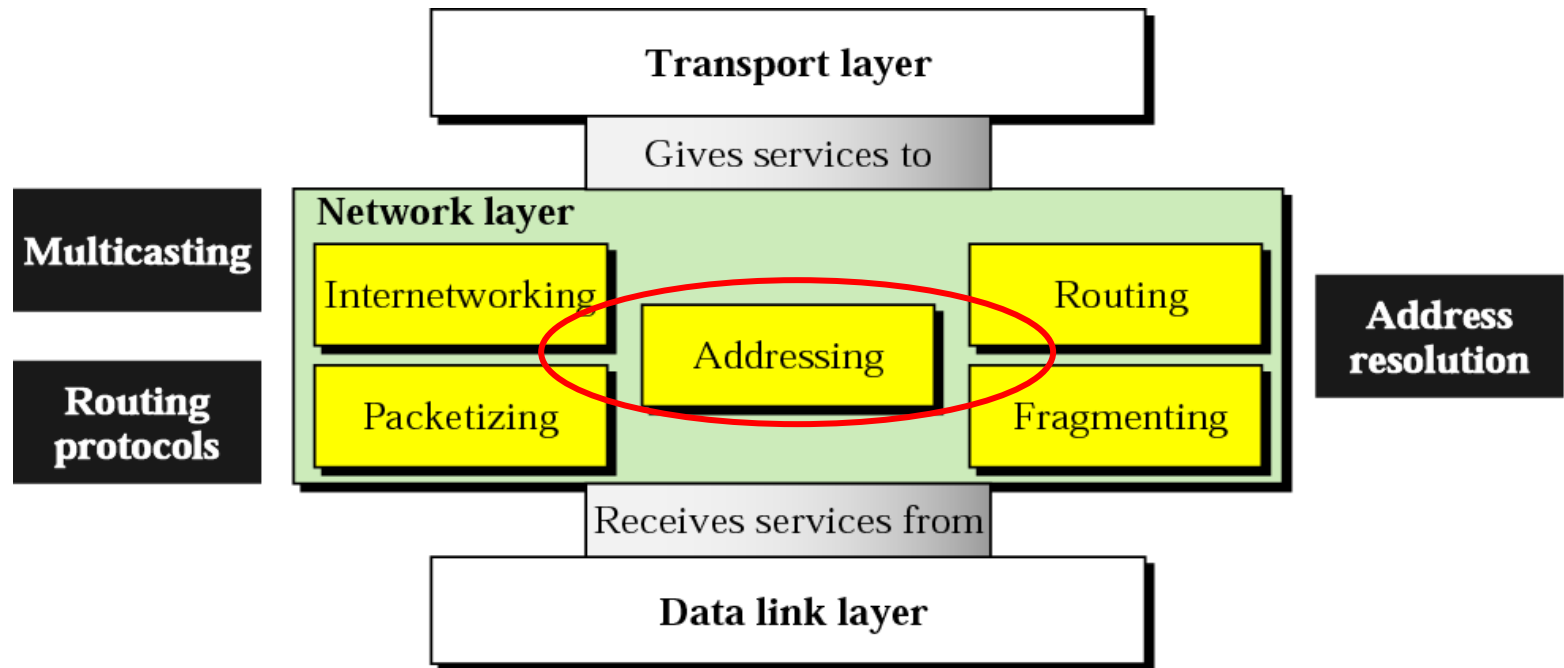


IP Fragmentation & Reassembly - example

- Original Datagram
 - 4000 byte data
 - 20 byte header
- MTU
 - 1420 byte (1st level)
 - 820 byte (2nd level)



Network layer



Addressing

- Hosts and routers connected to the network through an interface
- A host has one interface only
- A router has one interface for each network it interconnects
 - receives packet from one link on one interface and forwards it to another link on another interface
- IP address is associated with an interface rather than with a host or a router
- An IP address is a unique identifier for a node or host connection on an IP network. It is a part of scheme to identify hosts on the internet.

Addressing

- Need to uniquely identify each device on the Internet
 - analogy with the telephone system
 - two devices on the Internet (should) never have the same address
- IP address
 - 32 bit address (IPv4)
 - 128 bit address (IPv6)

IPv4 ADDRESSES

- *An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a device (for example, a computer or a router) to the Internet.*

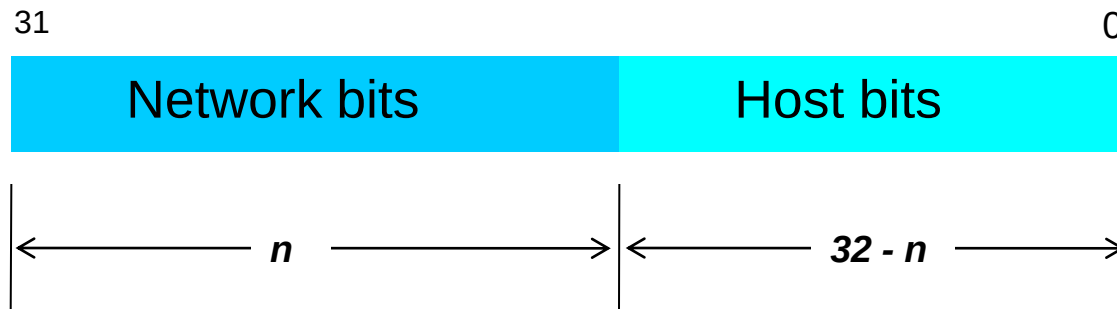
–An IPv4 address is 32 bits long.

–The IPv4 addresses are unique and universal.

–The address space of IPv4 is 2^{32} or 4,294,967,296.

Network and Host Portions

- some portion of the high-order bits represents the network address.
 - group of hosts that have identical bit patterns in the network address portion of their addresses.
- Number of bits used in host portion determines the number of hosts that we can have within the network.

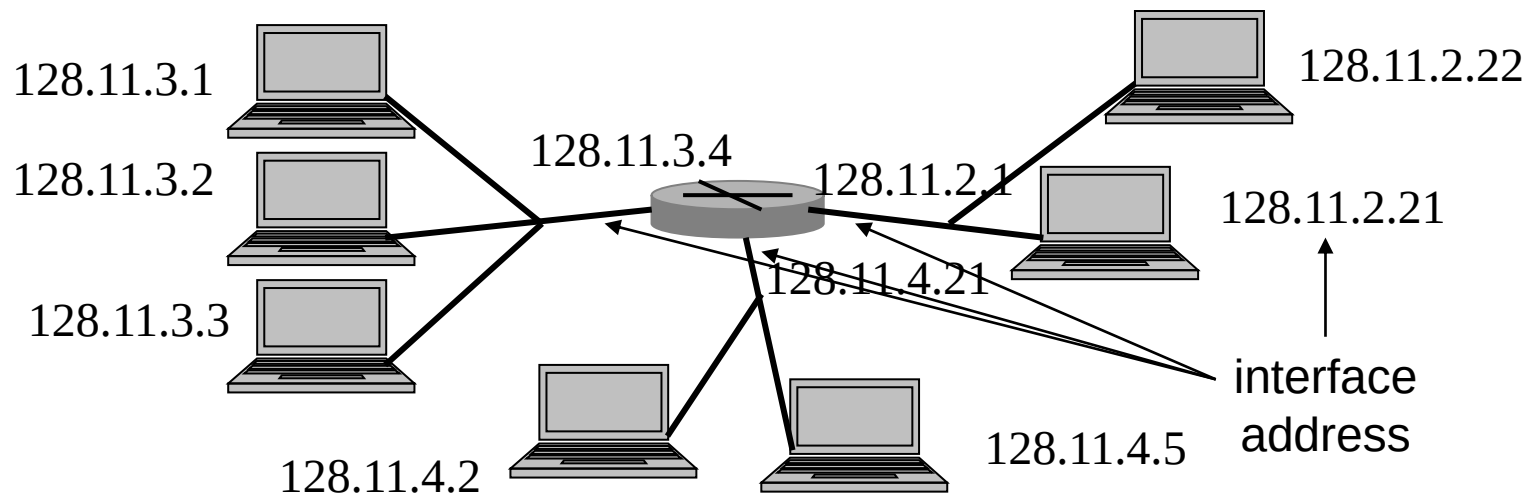
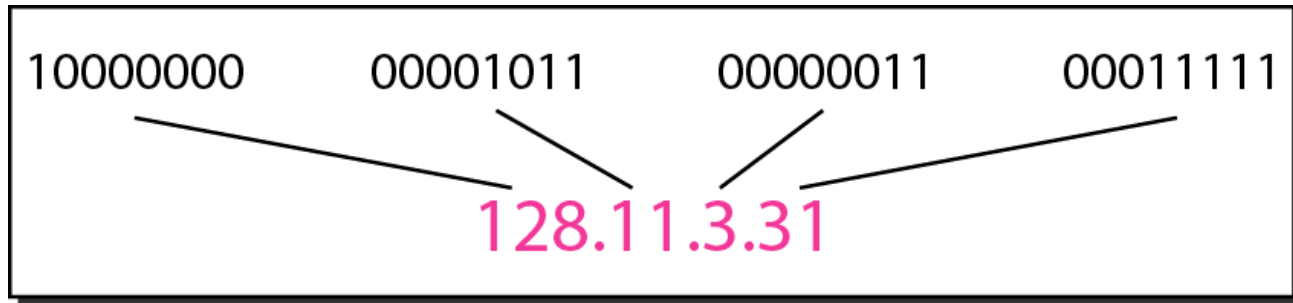


Types

- Network address - The address by which we refer to the network
- Broadcast address - A special address used to send data to all hosts in the network
- Host addresses - The addresses assigned to the end devices in the network

Notation for IPv4 address

- Binary notation
- Dotted-decimal notation



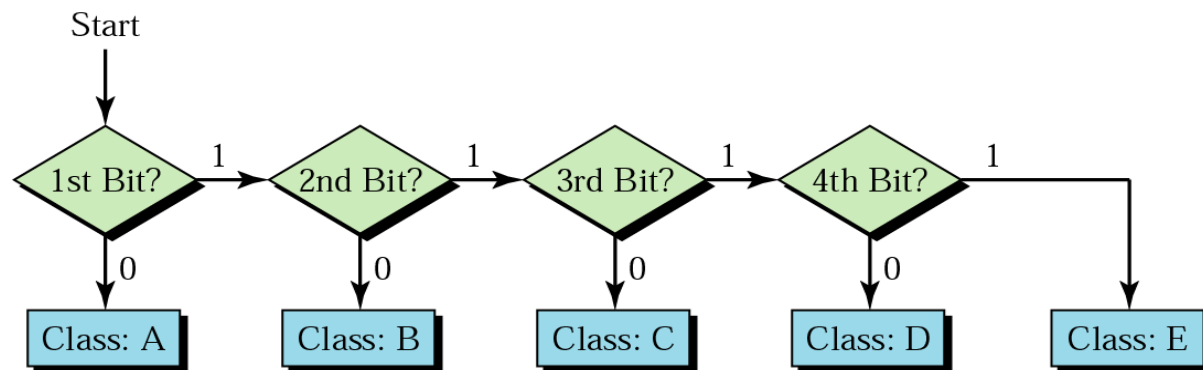
Addressing

1. Classful - obsolete
2. Classless

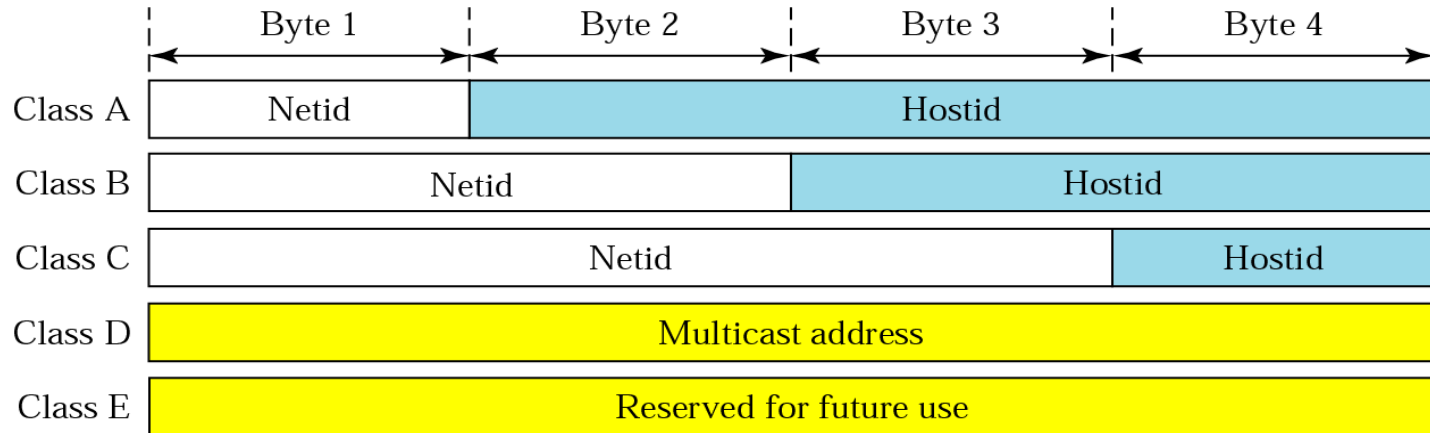
Classful (IP) addressing

		First byte	Second byte	Third byte	Fourth byte
unicast addresses	Class A	0 0-127			
	Class B	10 128-191			
	Class C	110 192-223			
multicast add.	Class D	1110 224-239			
reserved add.	Class E	1111 240-255			

- Based on the first few bits we can determine the class of address



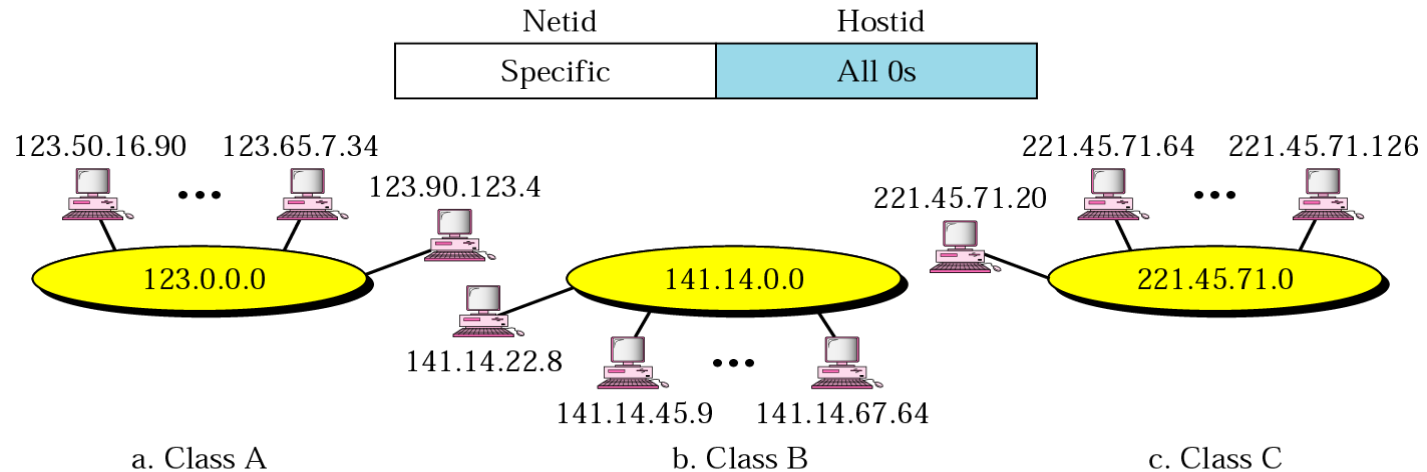
Netid & hostid



- Class A: 128 blocks with 16 777 216 addresses each -> **wasted!**
- Class B: 16 384 blocks with 65 536 addresses each -> **wasted!**
- Class C: 2 097 152 blocks with 256 addresses each -> **not enough**
- Class D: 1 block
- Class E: 1 block
- Classful addressing offers inefficient use of the address space
 - Example: Class B – 65K addresses may be assigned to an organization with 2K hosts

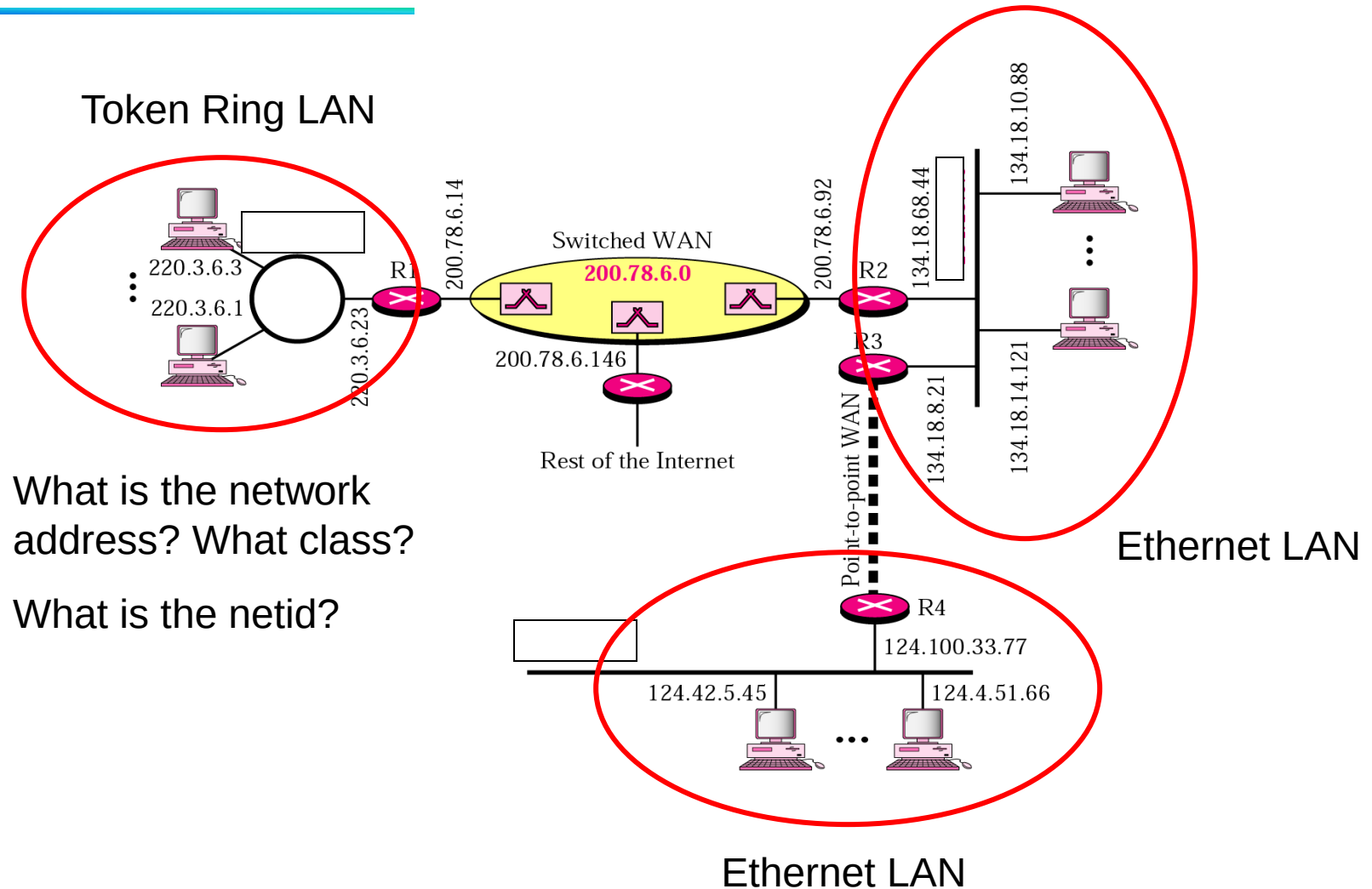
Network address

- Defines the network itself (cannot be assigned to a host)



- Properties
 - all host id are 0s
 - defines the network to the rest of the Internet
- What is the network definition now (from IP add. perspective)?

A simple internet with classful add.

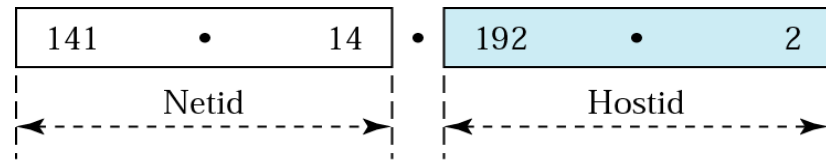


What is the network address? What class?

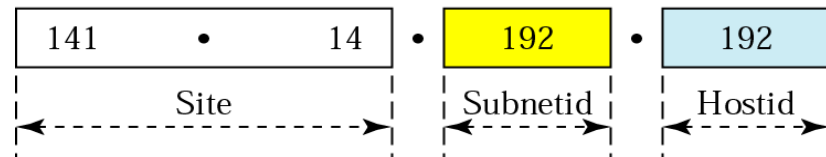
What is the netid?

Subnetting

- Dividing networks into smaller parts
 - more levels of hierarchy
- Hierarchy in addressing
 - Network (site)
 - subnetwork
 - host



a. Without subnetting



b. With subnetting

hierarchy in telephony

(040)	247	—	1000
-------	-----	---	------

area code exchange connection

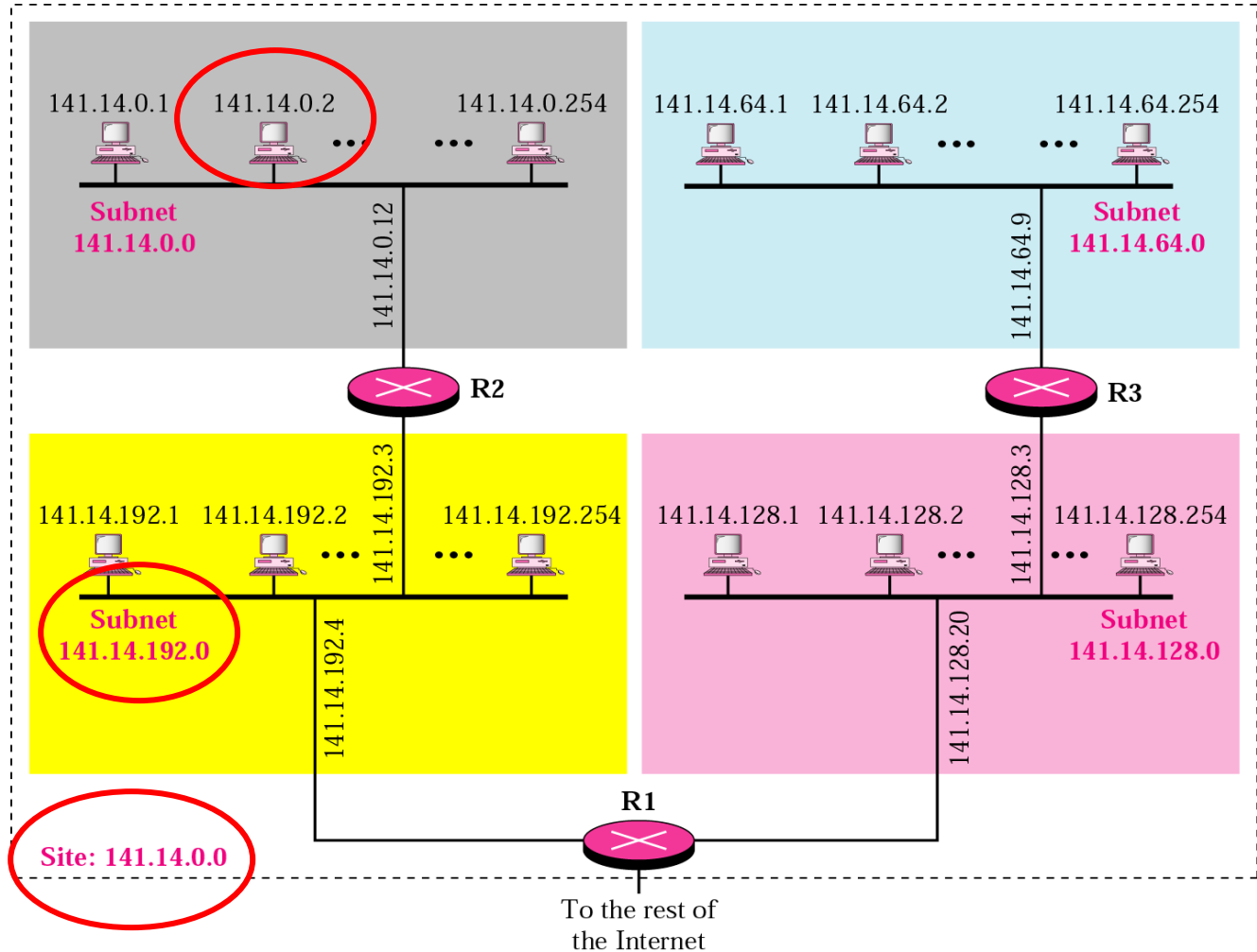
Example: Department based host grouping at the University

- The outside world “sees” one network only

Subnetting (cnt'd)

3 hierarchy levels

- Site
- Subnet
- Host



Addressing

-mask-

- Routing is based on both *network* and *subnetwork* addresses
 - Analogy: Parcel delivery → zip code and street address
- How can a router find the network or the subnetwork address to route the packet?
 - 1. Use default mask
 - 2. Use a subnet mask
- Default mask: 32-bit binary number ANDed with the address in the block
 1. if the bit in the mask = 1, then retain the bit in the address
 2. if the bit in the mask ≠ 1, then put 0

Class	In Binary	In Dotted-Decimal	Using Slash
A	11111111 00000000 00000000 00000000	255.0.0.0	/8
B	11111111 11111111 00000000 00000000	255.255.0.0	/16
C	11111111 11111111 11111111 00000000	255.255.255 .0	/24

number
of 1's

Addressing

-mask-

Example:

A router outside an organization receives a packet with the destination **190.240.7.91**. How it finds the network address to route the packet?

Solution:

1. First byte of the address defines a class. Class B.
2. The default mask for class B is **255.255.0.0**. The router ANDs this with the packet address to get **190.240.0.0**.
3. The router looks in the routing table to forward the packet to the appropriate network.

Q: How to find a destination within the network?

Addressing

- subnet mask -

A router inside an organization receives a packet with the destination **190.240.7.91**. How it finds the subnetwork address to route the packet?

Solution:

1. Assume the subnet mask is /19.
2. The router applies the mask to the address **190.240.7.91**.
Obtained subnet address is: **190.240.32.0**.
3. The router looks in the routing table to find how to route the packet to a destination.

Addressing

1. Classful
2. Classless – it is now in use

Classless addressing

- Solving problems with classful addressing:
 - $256 < \text{the number of IP addresses} < 16\,777\,216$
 - what if one needs at home 2 addresses only? 254 wasted?
- Solution: Classless addressing
 - addresses provided by an Internet Service Provider (ISP)
 - ISP divides blocks of addresses into groups of 2, 4, 8, 16...
- Variable-length blocks that belong to no class
 - the number of address block must be a power of 2
- Classless InterDomain Routing (CIDR)

Reserved Address

Address Block	Present Use	Reference
0.0.0.0/8	"This" Network	[RFC1700, page 4]
10.0.0.0/8	Private-Use Networks	[RFC1918]
14.0.0.0/8	Public-Data Networks	[RFC1700, page 181]
24.0.0.0/8	Cable Television Networks	--
39.0.0.0/8	Reserved but subject to allocation	[RFC1797]
127.0.0.0/8	Loopback	[RFC1700, page 5]
128.0.0.0/16	Reserved but subject to allocation	--
169.254.0.0/16	Link Local	--
172.16.0.0/12	Private-Use Networks	[RFC1918]
191.255.0.0/16	Reserved but subject to allocation	--
192.0.0.0/24	Reserved but subject to allocation	--
192.0.2.0/24	Test-Net	
192.88.99.0/24	6to4 Relay Anycast	[RFC3068]
192.168.0.0/16	Private-Use Networks	[RFC1918]
198.18.0.0/15	Network Interconnect Device Benchmark Testing	[RFC2544]
223.255.255.0/24	Reserved but subject to allocation	--
224.0.0.0/4	Multicast	[RFC3171]
240.0.0.0/4	Reserved for Future Use	[RFC1700, page 4]

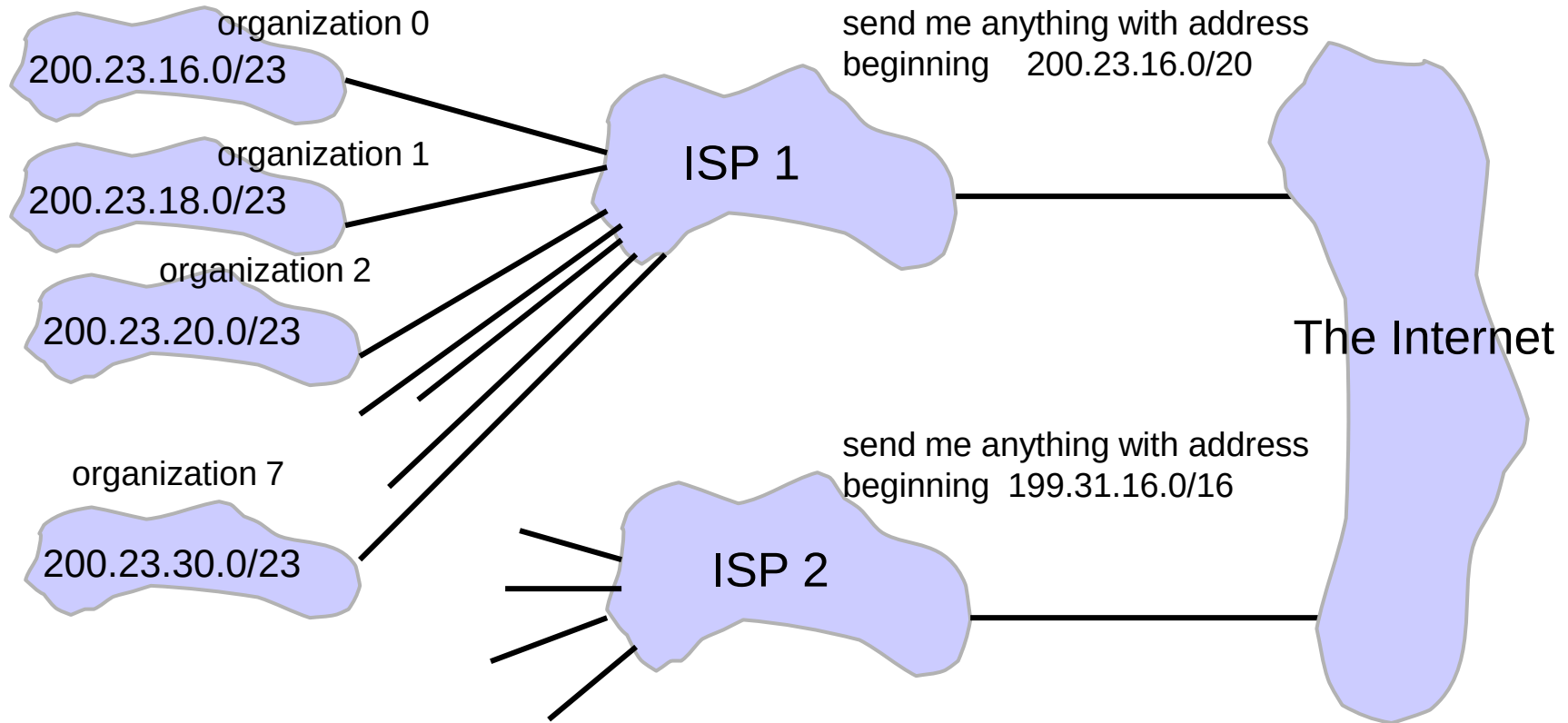
Obtaining a network address

- To obtain a block of IP addresses administrator might first contact its ISP
- ISP gives it the block from the larger block already allocated to ISP

Example (subnetting):

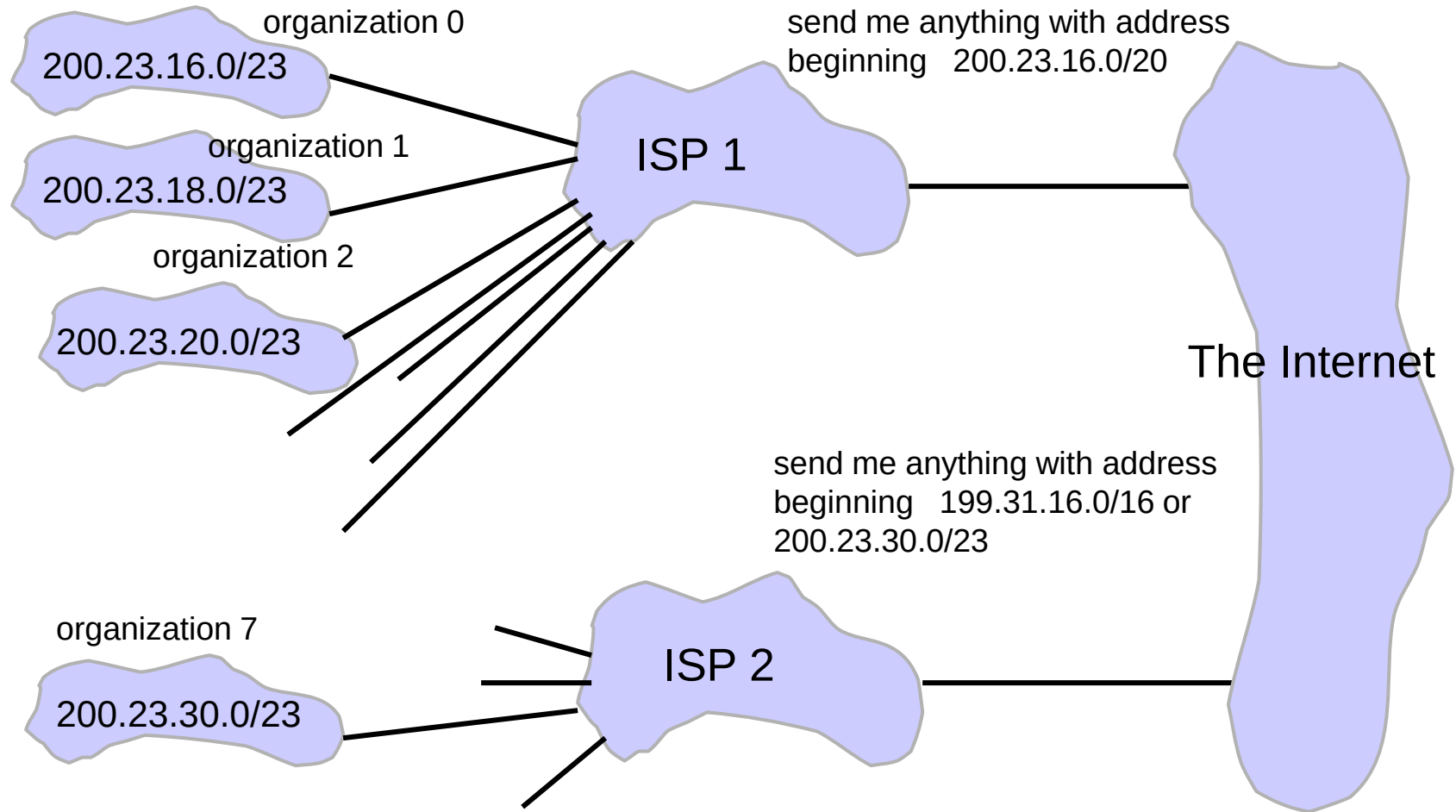
ISP's block	200.23.16.0/20	<u>11001000</u> <u>00010111</u> <u>00010000</u> 00000000
Organization 0	200.23.16.0/23	<u>11001000</u> <u>00010111</u> <u>00010000</u> 00000000
Organization 1	200.23.18.0/23	<u>11001000</u> <u>00010111</u> <u>00010010</u> 00000000
Organization 2	200.23.20.0/23	<u>11001000</u> <u>00010111</u> <u>00010100</u> 00000000
.		
.		
Organization 7	200.23.30.0/23	<u>11001000</u> <u>00010111</u> <u>00011110</u> 00000000

An example



- single network prefix is used to advertise multiple networks:
[route aggregation](#)

An example (cnt'd)



Some rules for IP addressing

1. The first address in the block can be found by setting the $32-n$ rightmost bits in the binary notation of the address to 0s.
 - Example: what is the first address in the block if one of the addresses is 205.16.37.39/28?
 - A: 11001101 00010000 00100101 00100111
11001101 00010000 00100101 00100000 -> 205.16.37.32
2. The last address in the block can be found by setting the rightmost $32-n$ bits to 1s.
 - what is the last address in the block if one of the addresses is 205.16.37.39/28?
 - A: 11001101 00010000 00100101 00100111
11001101 00010000 00100101 00101111 -> 205.16.37.47
3. The number of addresses in the block is 2^{32-n}
 - For the value of $n=28$ the number of addresses is 16.

Exercise

- An ISP is granted a block of addresses starting with 190.100.0.0/16 (How many addresses are available?). The ISP needs to distribute these addresses to 3 groups of customers:
 1. 1st group: 64 customers; each needs 256 addresses
 2. 2nd group: 128 customers; each needs 128 addresses
 3. 3rd group: 128 customers; each needs 128 addresses

Design the sub-blocks and find out how many addresses are available after these allocations.

Solution

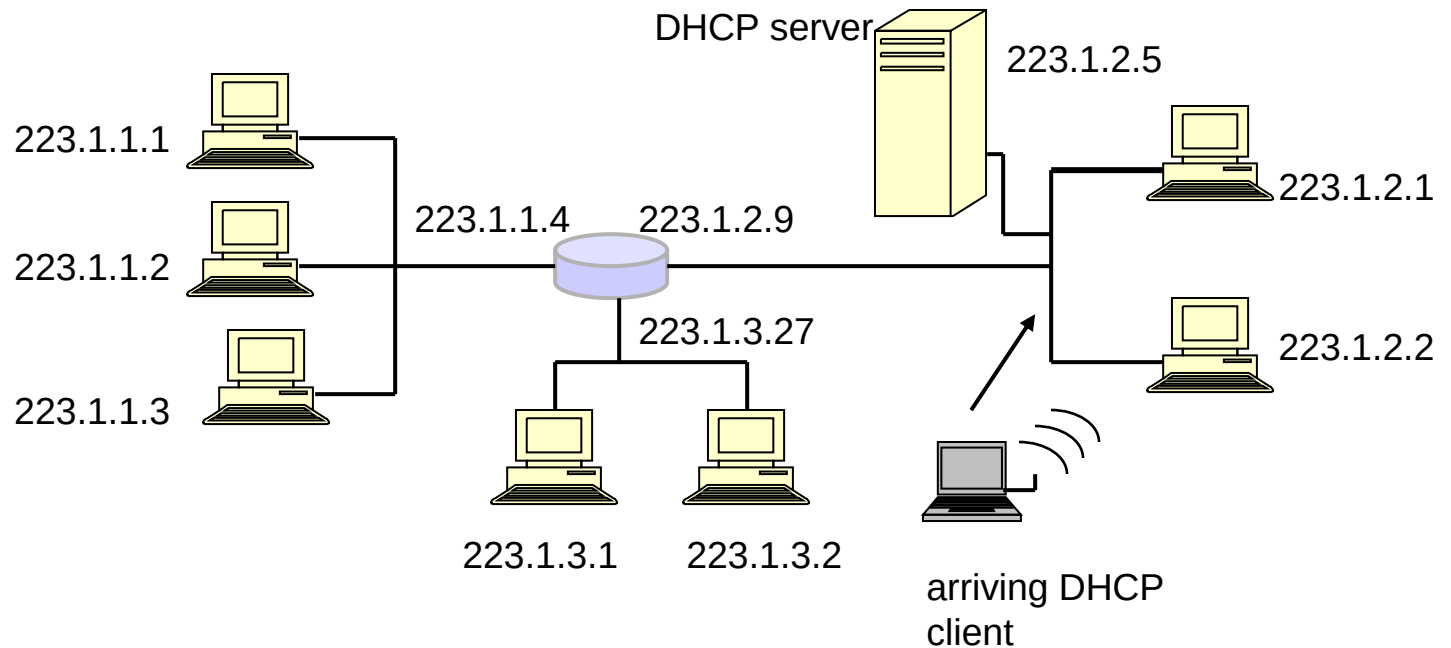
- Group1: 256 addresses-> 8 bits-> $32-8=24$ bits for the mask
 - 1st customer: 190.100.0.0/24 – 190.100.0.255/24
 - 2nd customer: 190.100.1.0/24 – 190.100.1.255/24
 - 64th customer: 190.100.63.0/24 – 190.100.63.255/24
 - Total: $64 \times 256 = 16\ 384$
- Group2: 128 addresses->7 bits-> $32-7=25$ mask bits
 - 1st customer: 190.100.64.0/25 – 190.100.64.127/25
 - 2nd customer: 190.100.64.128/25 – 190.100.64.255/25
 - 128th customer: 190.100.127.128/25 – 190.100.127.255/25
 - Total: $128 \times 128 = 16\ 384$
- ...

Obtaining a host address

- Manual configuration
 - put an IP address in the file
- Dynamic Host Configuration Protocol (DHCP)
 - IP assigned automatically
 - host learns about its subnet mask and IP of both the DNS server & the first-hop router
 - very useful when hosts are frequently joining & leaving network
 - dormitories, classrooms, libraries...
 - address assigned on a temporarily basis
 - 2000 hosts in total; 400 hosts on line -> 512 IP addresses are sufficient

DHCP

- a client-server protocol
- client: typically a newly arriving host



DHCP (cnt'd)

- Host knows neither the IP address of the network it wants to attach to, nor the IP add. of the DNS server
1. DHCP server discovery
 - broadcast DHCP discovery message (sent within UDP on port 67)
 - destination address 255.255.255.255
 - source address 0.0.0.0
 2. DHCP server offers
 - proposed IP address, network mask, IP address lease time
 3. DHCP request
 4. DHCP ACK

IP addressing: the last word...

Q: How does an ISP get block of addresses?

A: **ICANN**: Internet **C**orporation for **A**ssigned
Names and **N**umbers

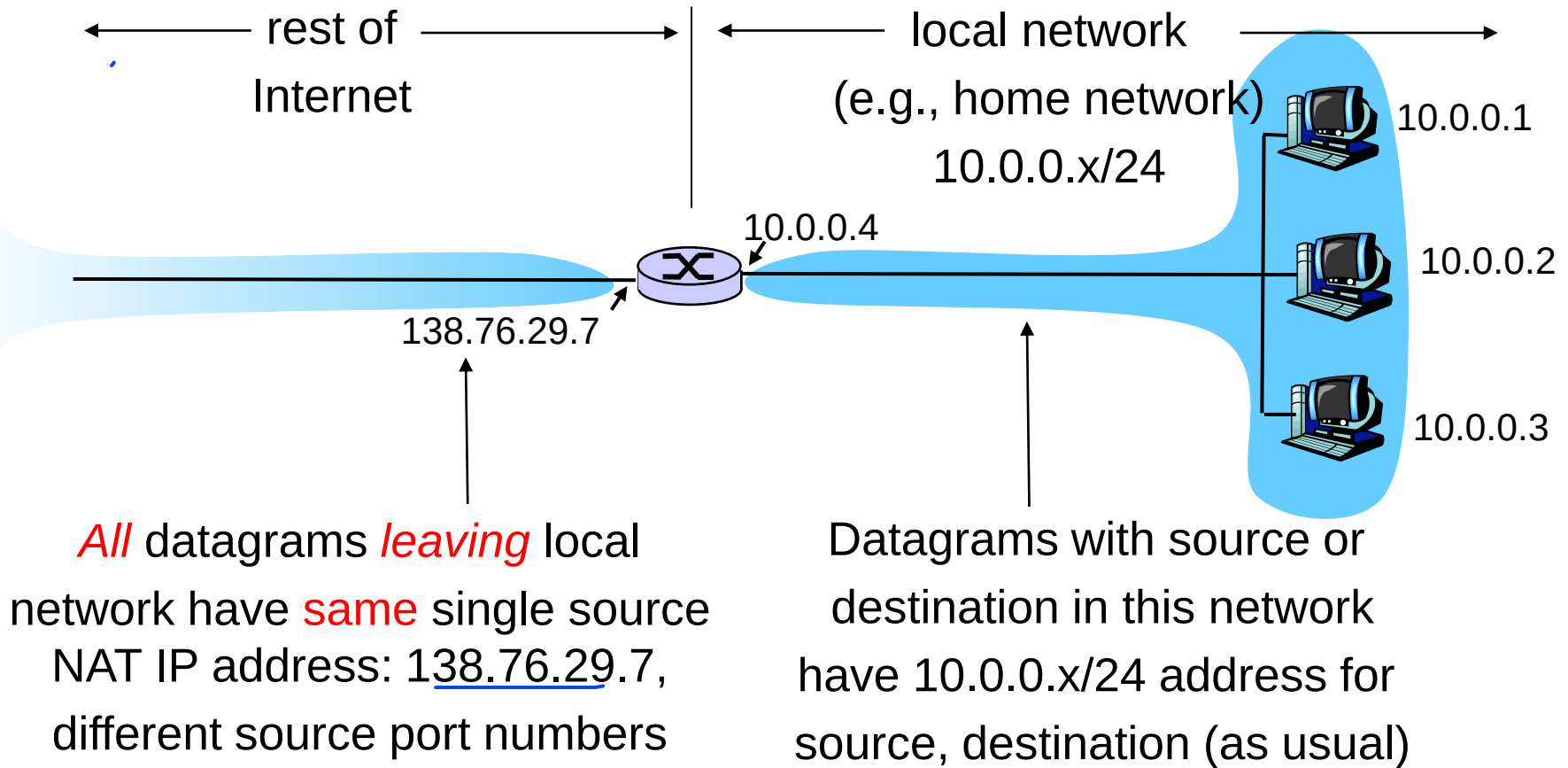
- allocates addresses
- manages DNS
- assigns domain names, resolves disputes

Private address and NAT,PAT

- IPv4 address space limited
- Solution: NAT,PAT
- Private address
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16

<i>Range</i>			<i>Total</i>
10.0.0.0	to	10.255.255.255	2^{24}
172.16.0.0	to	172.31.255.255	2^{20}
192.168.0.0	to	192.168.255.255	2^{16}

PAT: Port Address Translation



PAT: Port Address Translation

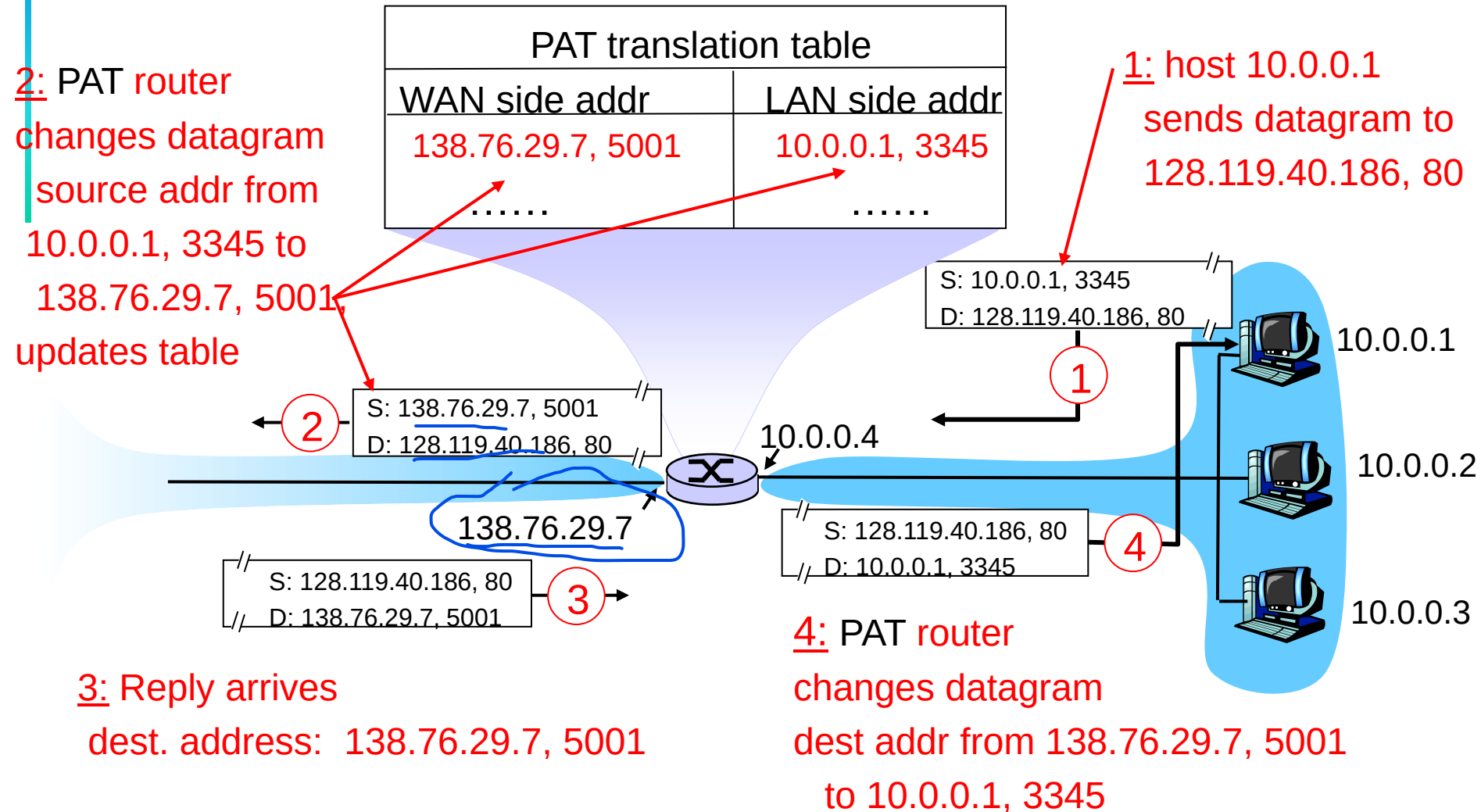
- **Motivation:** local network uses just one IP address as far as outside world is concerned:
 - range of addresses not needed from ISP: just one IP address for all devices
 - can change addresses of devices in local network without notifying outside world
 - can change ISP without changing addresses of devices in local network
 - devices inside local network not explicitly addressable, visible by outside world (a security plus).

PAT: Port Address Translation

Implementation: PAT router must:

- *outgoing datagrams: replace* (source IP address, port #) of every outgoing datagram to (PAT IP address, new port #)
... remote clients/servers will respond using (PAT IP address, new port #) as destination addr.
- *remember (in PAT translation table)* every (source IP address, port #) to (PAT IP address, new port #) translation pair
- *incoming datagrams: replace* (PAT IP address, new port #) in dest fields of every incoming datagram with corresponding (source IP address, port #) stored in PAT table

PAT : Port Address Translation



PAT : Port Address Translation

- 16-bit port-number field:
 - 60,000 simultaneous connections with a single LAN-side address!
- NAT,PAT is controversial:
 - routers should only process up to layer 3
 - violates end-to-end argument
 - NAT possibility must be taken into account by app designers, eg, P2P applications
 - address shortage should instead be solved by IPv6

Pros & Cons of PAT

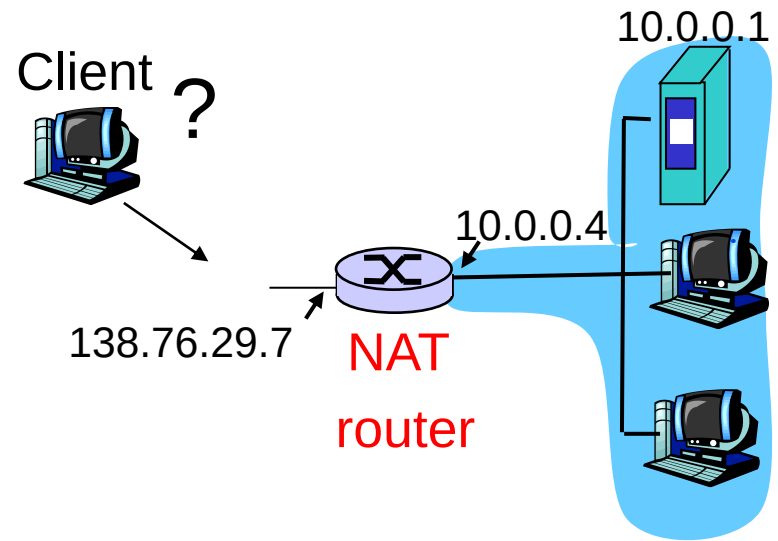
- **PAT PROs**
- Use of a single registered IP address for an entire network
- Independence of ISP IP addresses
- Transparent to end systems in some cases (increased security)
- Delays need for IPv4 replacement
- Mask the true internal IP addresses of the internal network

Pros & Cons of PAT

- **PAT CONS**
- Breaks end-to-end model (the private IP space might already be in use)
- Enables end-to-end address conflicts (encourages poor address management)
- Increases local support burden and complexity
- Certain applications do not work properly in conjunction
 - Certain PAT boxes don't allow member NT servers to talk to a PDC
 - Applications needing encryption and key exchange might be problematic
- Increases the probability of mis-addressing
- It might require more work if assigning registered IPs to private IPs
- Possible performance degradation if you get close to the limit of about 25,000 connections
- Obscures the ability to manage from the public side of the NAT box.

PAT traversal problem

- client wants to connect to server with address 10.0.0.1
 - server address 10.0.0.1 local to LAN (client can't use it as destination addr)
 - only one externally visible PATted address: 138.76.29.7
- solution 1: statically configure PAT to forward incoming connection requests at given port to server
 - e.g., (138.76.29.7, port 2500) always forwarded to 10.0.0.1 port 25000

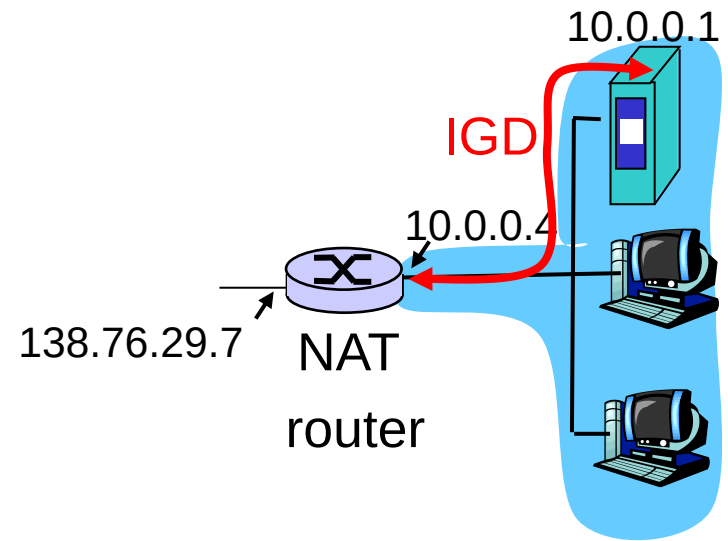


PAT traversal problem

- **solution 2:** Universal Plug and Play (UPnP) Internet Gateway Device (IGD) Protocol. Allows PAT ted host to:

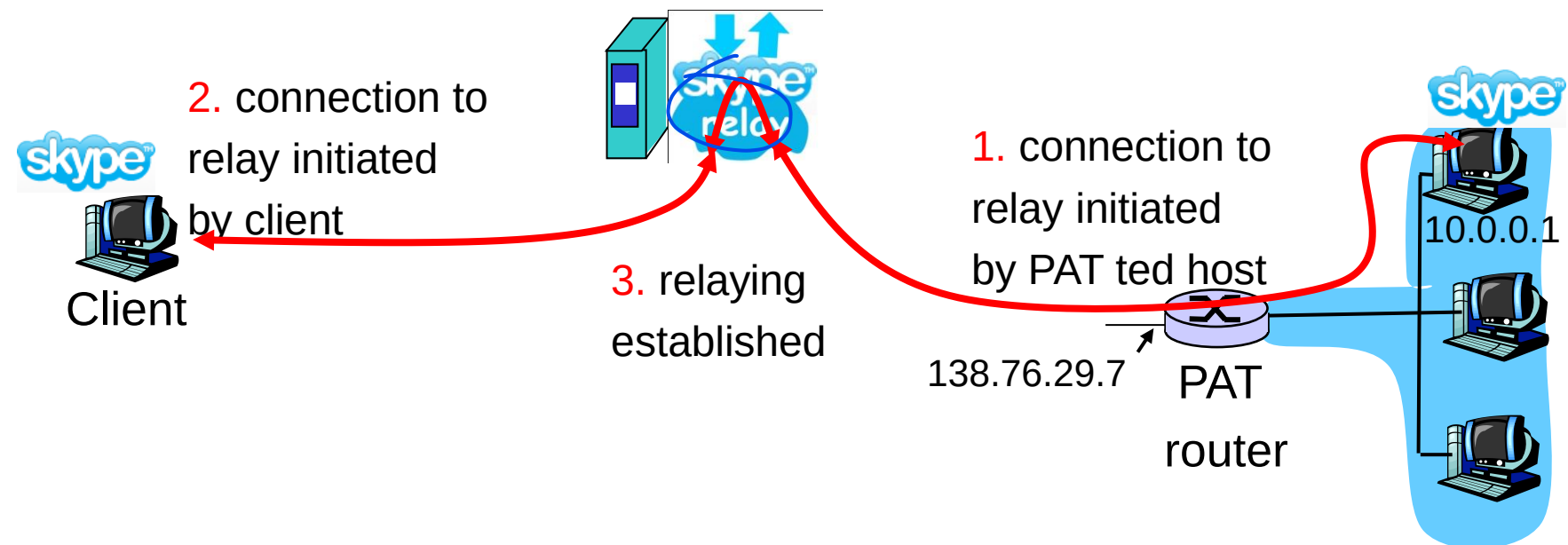
- ❖ learn public IP address (138.76.29.7)
- ❖ add/remove port mappings (with lease times)

i.e., automate static PAT port map configuration

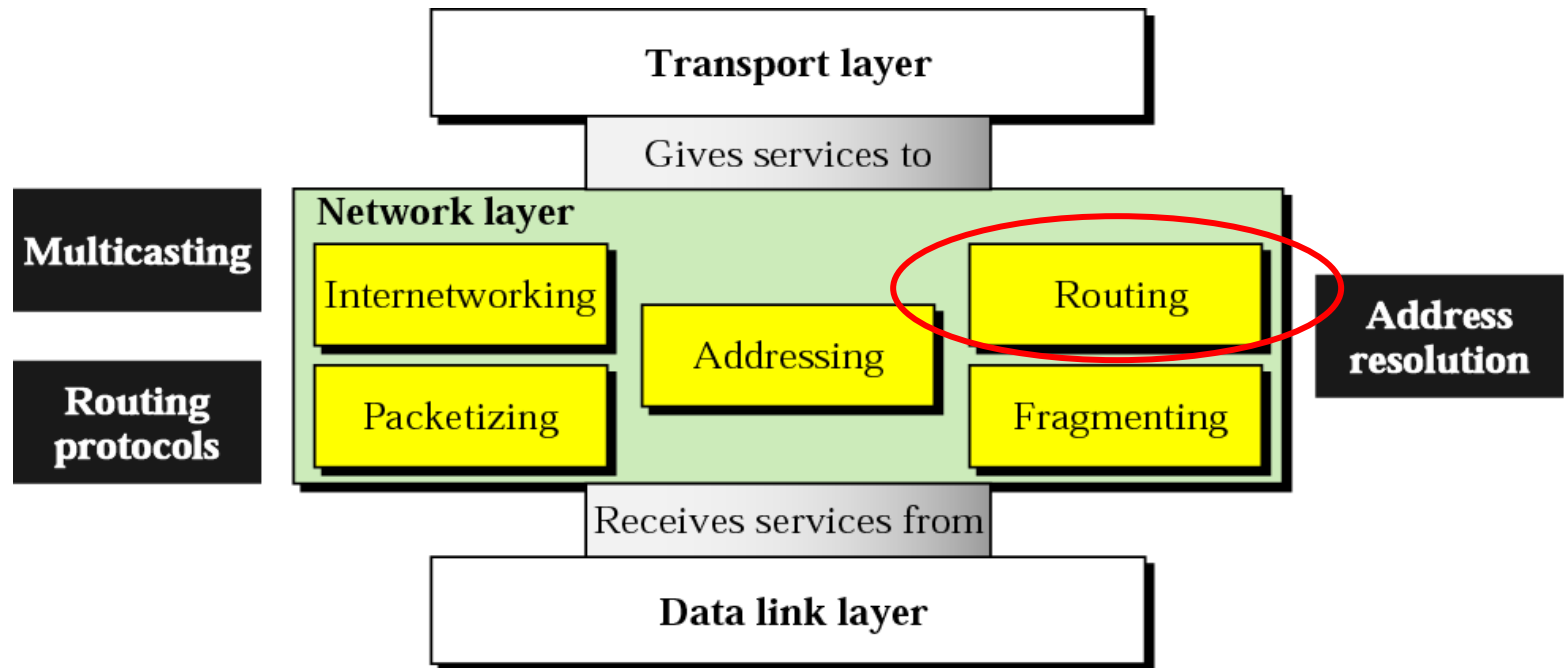


PAT traversal problem

- **solution 3: relaying (used in Skype)**
 - PATed client establishes connection to relay
 - External client connects to relay
 - relay bridges packets between to connections



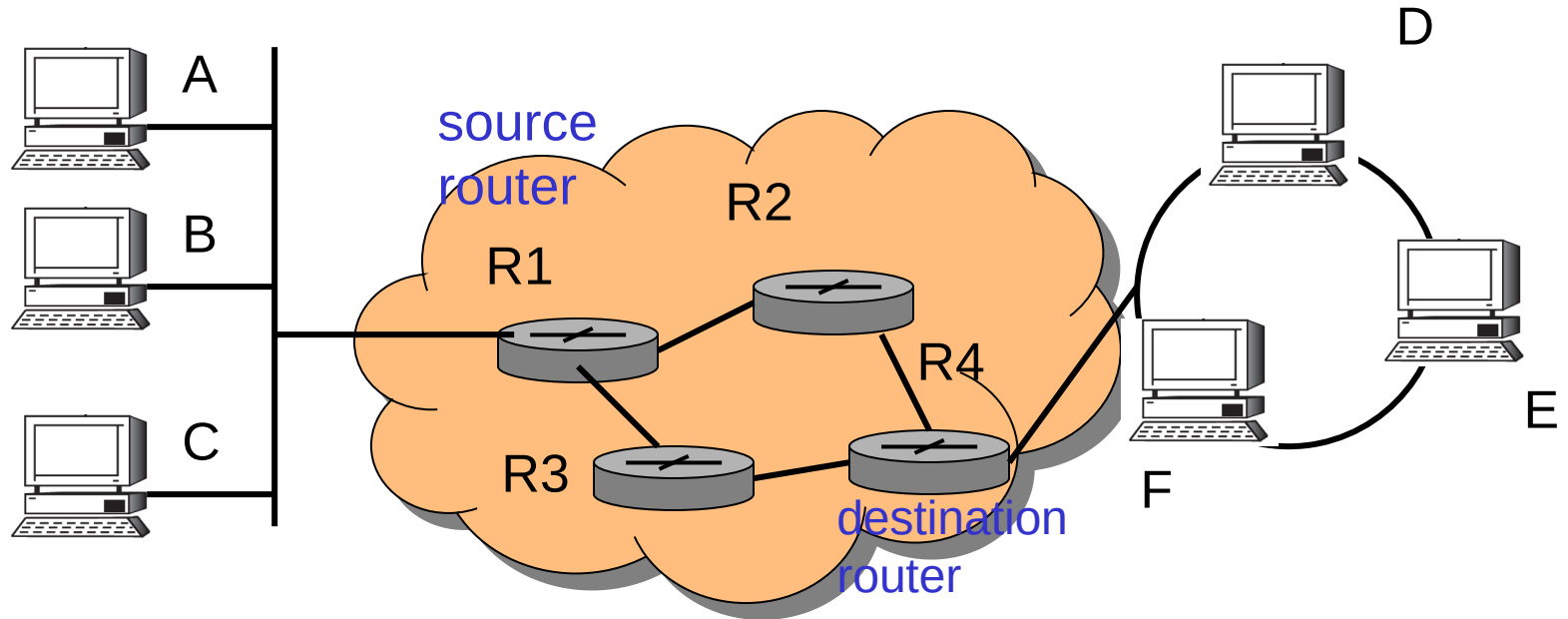
Network layer



Routing & forwarding

- Not the same thing!
- Routing- filling in the routing tables
- Forwarding – handling the packets based on the routing tables

Routing algorithm



- How to find the best path from A to F?
- How does R1 chooses the best route to R4?
- “A part of the network software responsible for deciding which output line an incoming packet should be transmitted on.”

Routing Algorithm classification

- DYNAMIC
 - change routing decisions to reflect changes in the topology
 - adapt for changes in the traffic (load change)
 - routes change more quickly
 - ALGORITHMS: where the routers get the information from?
 - locally
 - from adjacent routers
 - from all routers
 - ALGORITHMS: when they change their routes?
 - every ΔT sec
 - when the load changes
 - when topology changes
- STATIC
 - routes computed in advance
 - node failures, current load etc. not taken into account
- Note that both dynamic & static algorithms can be either *load (congestion) - sensitive* or *load (congestion) -insensitive*.

Routing Algorithm classification

1. Global routing algorithm

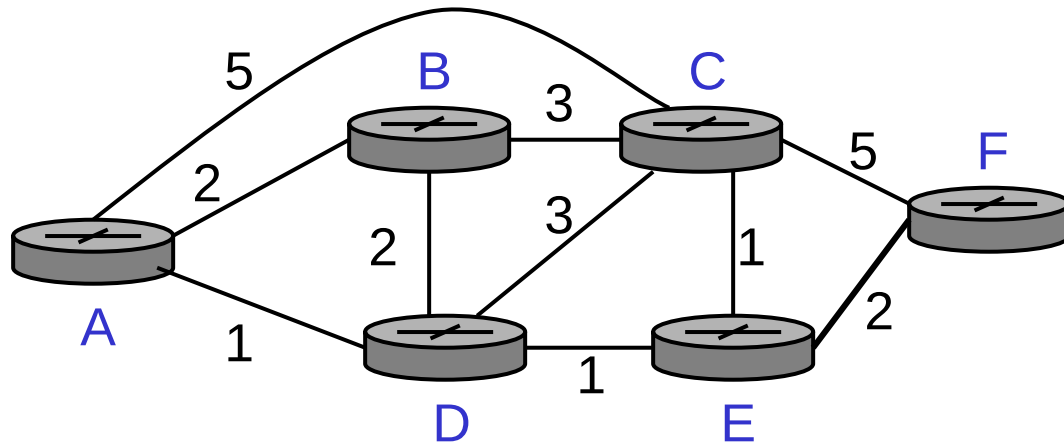
- least-cost path calculated using global knowledge about network
- all routers have complete topology, link cost info
- **input:** connectivity between all nodes & link costs
- link state algorithms

2. Decentralized routing algorithm

- least-cost path calculated in an iterative, distributed manner
- no node has complete info about the costs of all network links
- begins with a cost of the directly attached links
- info exchange with the neighbouring nodes
- distance vector algorithms

Determining the path

- Build a graph of the subnet:
 - each router represented by a node
 - node connected by a link (communication line)



- cost: number of hops, geographic distance in km, queuing delay, transmission delay, bandwidth, reliability, price
- least-cost path – the minimum sum of the cost of the links
- shortest path – crossing the smallest number of links

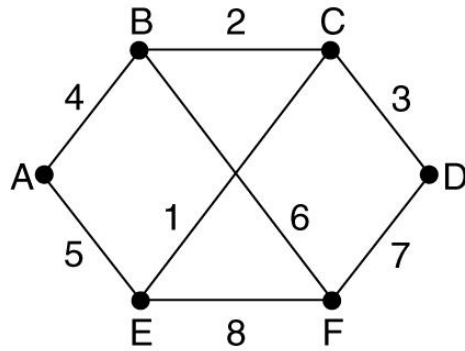
Difference Between Link State and Distance Vector Routing Protocol

Particular	Distance Vector Routing Protocol	Link State Routing Protocol
Example of Algorithm	RIP, IGRP,EIGRP Bellmon-ford	OSPF, IS-IS Shortest Path first (SPF)
Support subnet	Only classfull Routing(In case of RIPv2 & EIGRP also classless)	Classfull, Classless, VLSM, Summarization
Scale	Small, limited hop	Large
Table creation	Only Routing Table	Routing Table, Neighbor Table and Topology Table
Convergence time	Very Slow	Fast
Updating	On Broadcast	On multicast
Updating based on	Rumor	Based on topology table
Updating time	When periodic timer expired	Whenever changing occurs
Updating contents	Whole routing table	Only changed information
HOP	Limited	Unlimited
Needs Memory	Less	High
CPU Cycle	Less	High
Configuration	Simple	Advanced
Risk of Layer 3 Loop	Yes	No
Hierarchical Structure	No	Yes
OPEN Standard	Yes	Yes

Link state routing

- A Global routing algorithm
- Each router does the following (repeatedly):
 - discover neighbors
 - A router learns about its neighbours by sending a special HELLO packet to each point-to-point line.
 - measure the cost to each neighbor
 - construct a link state packets
 - send this packet to all other routers
 - compute *locally* the shortest path to every other router when this information is received
 - Use Dijkstra's algorithm

Constructing link state packets



(a)

subnet

sender		Link		State		Packets	
A		B		C		D	
Seq.		Seq.		Seq.		Seq.	
Age		Age		Age		Age	
B	4	A	4	B	2	C	3
E	5	C	2	D	3	F	7
		F	6	E	1		

(b)

link state packets for this subnet

- When to build these packets?
 - at regular time intervals,
 - on occurrence of some significant event,
 - link goes down (or comes back); cost change appreciably.

Distributing the link state packets

- Typically, flooding
 - routers recognize packets passed earlier
 - sequence number incremented for each new packet sent
 - routers keep track of the (source router, sequence) pair
 - thus avoiding the exponential packet explosion

A Link-State Routing Algorithm

Dijkstra's algorithm

- computes least cost paths from one node ('source') to all other nodes
 - gives forwarding table for that node
- iterative: after k iterations, know least cost path to k dest.'s

Notation:

- $c(x,y)$: link cost from node x to y ; $= \infty$ if not direct neighbors
- $D(v)$: current value of cost of path from source to dest. v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least cost path definitively known

Dijkstra's Algorithm

1 **Initialization:**

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent to u

5 then $D(v) = c(u,v)$

6 else $D(v) = \infty$

7

8 **Loop**

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 $D(v) = \min(D(v), D(w) + c(w,v))$

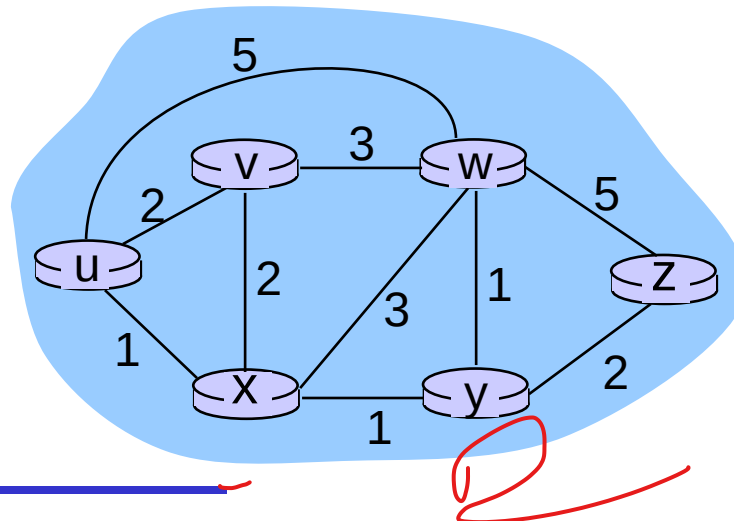
13 /* new cost to v is either old cost to v or known

14 shortest path cost to w plus cost from w to v */

15 **until all nodes in N'**

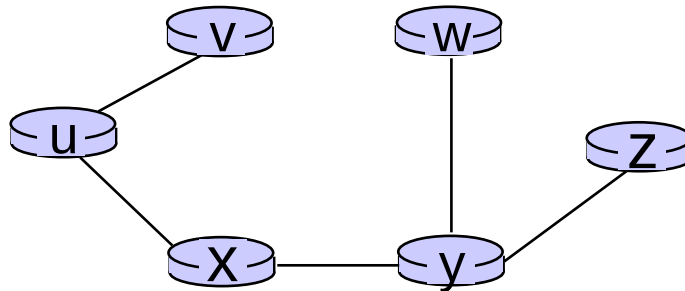
Dijkstra's algorithm: example

Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x	1,u	2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



Dijkstra's algorithm: example (2)

Resulting shortest-path tree from u:



Resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

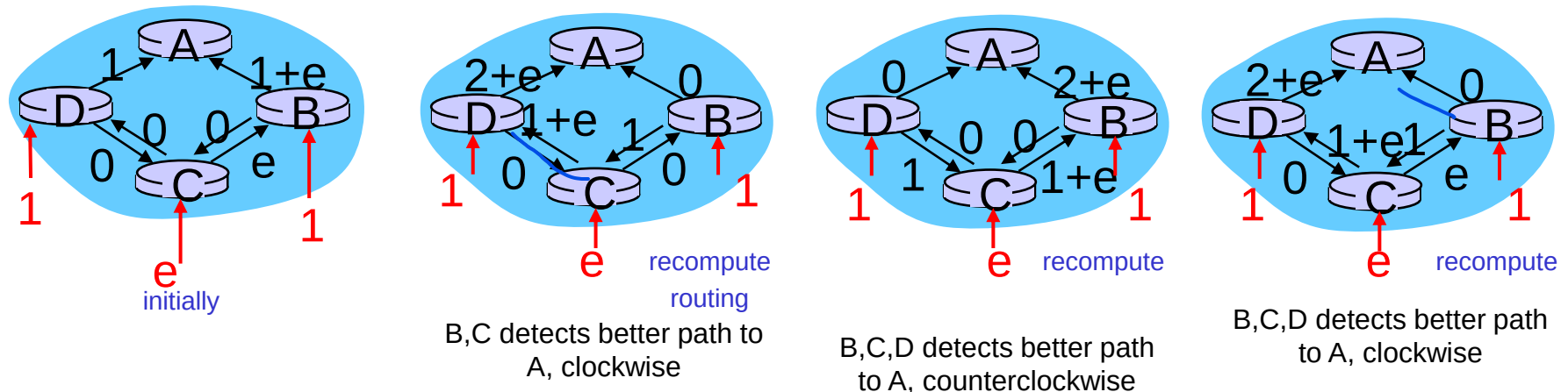
Dijkstra's algorithm, discussion

Algorithm complexity: n nodes

- each iteration: need to check all nodes, w , not in N'
- $n(n+1)/2$ comparisons, worst case complexity: $O(n^2)$
>order of n squared
- more efficient implementations possible: $O(n \log n)$

Oscillations possible:

- e.g., link cost = amount of carried traffic



The Distance Vector Routing

- Dynamic algorithm
 - takes current network condition into account
- Distributed
 - each node receives some information from one or more of its *directly attached* neighbors, performs a calculation, and then distributes the results of its calculation back to its neighbors.
- Iterative
 - algorithm performed in steps until no more information to exchange
 - initially, each node knows only about its adjacent nodes
- Asynchronous
 - nodes do not operate in lockstep with each other
 - local link cost change
 - DV update message from neighbor

The Distance Vector Routing

- Assume node X routes to Y via node Z directly attached to X(a neighbor)
- The distance to Y is updated by **Bellman-Ford Equation** (dynamic programming)

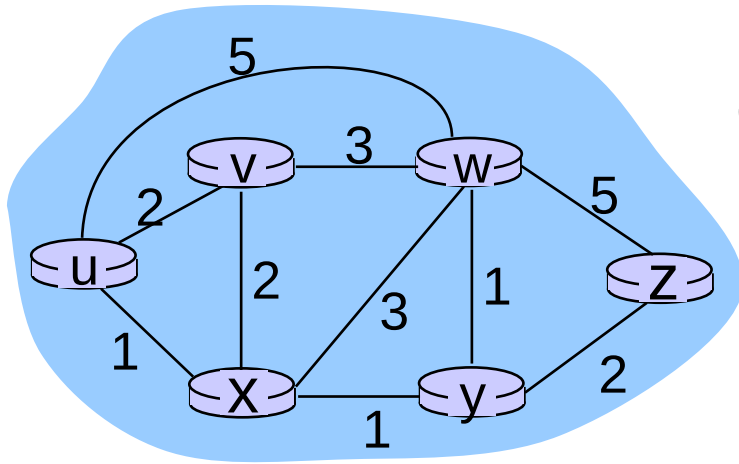
$$D_X(Y) = \min_{Z \in N(X)} (c(X, Z) + D_Z(Y))$$

Where $D_X(Y)$ = current least cost distance from X to Y

$N(X)$ = neighbors of node X (including X)

$c(X, Z)$ = the distance of the direct link between X and Z

Bellman-Ford example



Clearly, $D_v(z) = 5$, $D_x(z) = 3$, $D_w(z) = 3$

B-F equation says:

$$\begin{aligned} D_u(z) &= \min \{ c(u,v) + D_v(z), \\ &\quad c(u,x) + D_x(z), \\ &\quad c(u,w) + D_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Node that achieves minimum is next
hop in shortest path → forwarding table

Distance Vector Algorithm

At each node, x :

1 **Initialization:**

2 for all destinations y in N :

3 $D_x(y) = c(x,y)$ /* if y is not a neighbor then $c(x,y) = \infty$ */

4 for each neighbor w

5 $D_w(y) = ?$ for all destinations y in N

6 for each neighbor w

7 send distance vector $\mathbf{D}_x = [D_x(y): y \text{ in } N]$ to w

8

9 **loop**

10 **wait** (until I see a link cost change to some neighbor w or

11 until I receive a distance vector from some neighbor w)

12

13 for each y in N :

14 $D_x(y) = \min_v \{c(x,v) + D_v(y)\}$

15

16 **if** $D_x(y)$ changed for any destination y

17 send distance vector $\mathbf{D}_x = [D_x(y): y \text{ in } N]$ to all neighbors

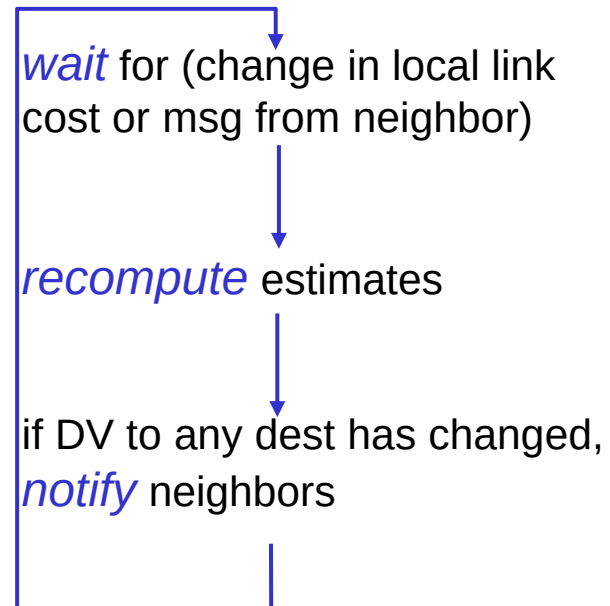
18

19 **forever**

Distance Vector Algorithm

- Each node x maintains
 - distance vector $\mathbf{D}_x = [D_x(y): y \in N(x), \text{NextHop}]$
 - also maintains its neighbors' distance vectors
- From time-to-time, each node sends its own distance vector estimate to neighbors - Asynchronous
- When a node x receives new DV estimate from neighbor, it updates its own DV using B-F equation:
- if any changes in DV, *notify* neighbors

At Each node:



$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = \underline{\underline{2}}$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

node x table

		cost to		
from		x	y	z
	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

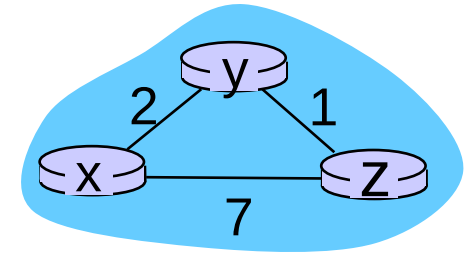
		cost to		
from		x	y	z
	x	0	2	3
	y	2	0	1
	z	7	1	0

node y table

		cost to		
from		x	y	z
	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

node z table

		cost to		
from		x	y	z
	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0



time

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min\{2+1, 7+0\} = 3$$

node x table

		cost to		
from		x	y	z
	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

node y table

		cost to		
from		x	y	z
	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

node z table

		cost to		
from		x	y	z
	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

		cost to		
from		x	y	z
	x	0	2	3
	y	2	0	1
	z	7	1	0

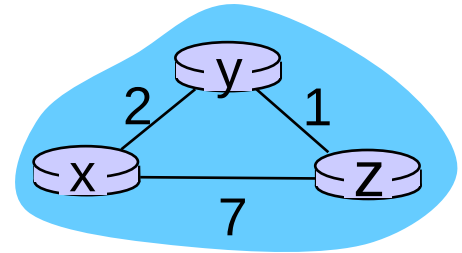
		cost to		
from		x	y	z
	x	0	2	7
	y	2	0	1
	z	7	1	0

		cost to		
from		x	y	z
	x	0	2	7
	y	2	0	1
	z	3	1	0

		cost to		
from		x	y	z
	x	0	2	3
	y	2	0	1
	z	3	1	0

		cost to		
from		x	y	z
	x	0	2	3
	y	2	0	1
	z	3	1	0

		cost to		
from		x	y	z
	x	0	2	3
	y	2	0	1
	z	3	1	0

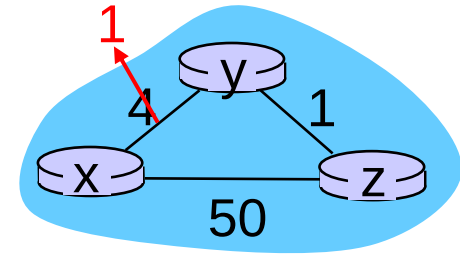


time →

Distance Vector: link cost changes

Link cost changes:

- ❑ node detects local link cost change
- ❑ updates routing info, recalculates distance vector
- ❑ if DV changes, notify neighbors



“good
news
travels
fast”

At time t_0 , y detects the link-cost change, updates its DV, and informs its neighbors.

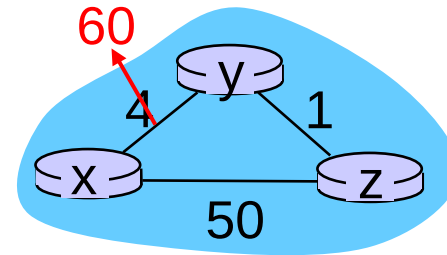
At time t_1 , z receives the update from y and updates its table. It computes a new least cost to x and sends its neighbors its DV.

At time t_2 , y receives z 's update and updates its distance table. y 's least costs do not change and hence y does *not* send any message to z .

Distance Vector: link cost changes

Link cost changes:

- ❑ good news travels fast
- ❑ bad news travels slow - “count to infinity” problem!
- ❑ 44 iterations before algorithm stabilizes: see text



Poisoned reverse:

- ❑ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❑ will this completely solve count to infinity problem?

Comparison of LS and DV algorithms

- **Message complexity**
- LS: with n nodes, E links, $O(nE)$ msgs sent
- DV: exchange between neighbors only
 - convergence time varies
- **Speed of Convergence**
- LS: $O(n^2)$ algorithm requires $O(nE)$ msgs
 - may have oscillations
- DV: convergence time varies
 - may be routing loops
 - count-to-infinity problem

Robustness: what happens if router malfunctions?

LS:

node can advertise incorrect *link* cost
each node computes only its *own* table

DV:

DV node can advertise incorrect *path* cost
each node's table used by others
error propagate thru network

Hierarchical Routing

Our routing study thus far - idealization

- all routers identical
- network “flat”

... *not* true in practice

scale: with 200 million destinations:

- can't store all dest's in routing tables!
- routing table exchange would swamp links!

administrative autonomy

- internet = network of networks
- each network admin may want to control routing in its own network

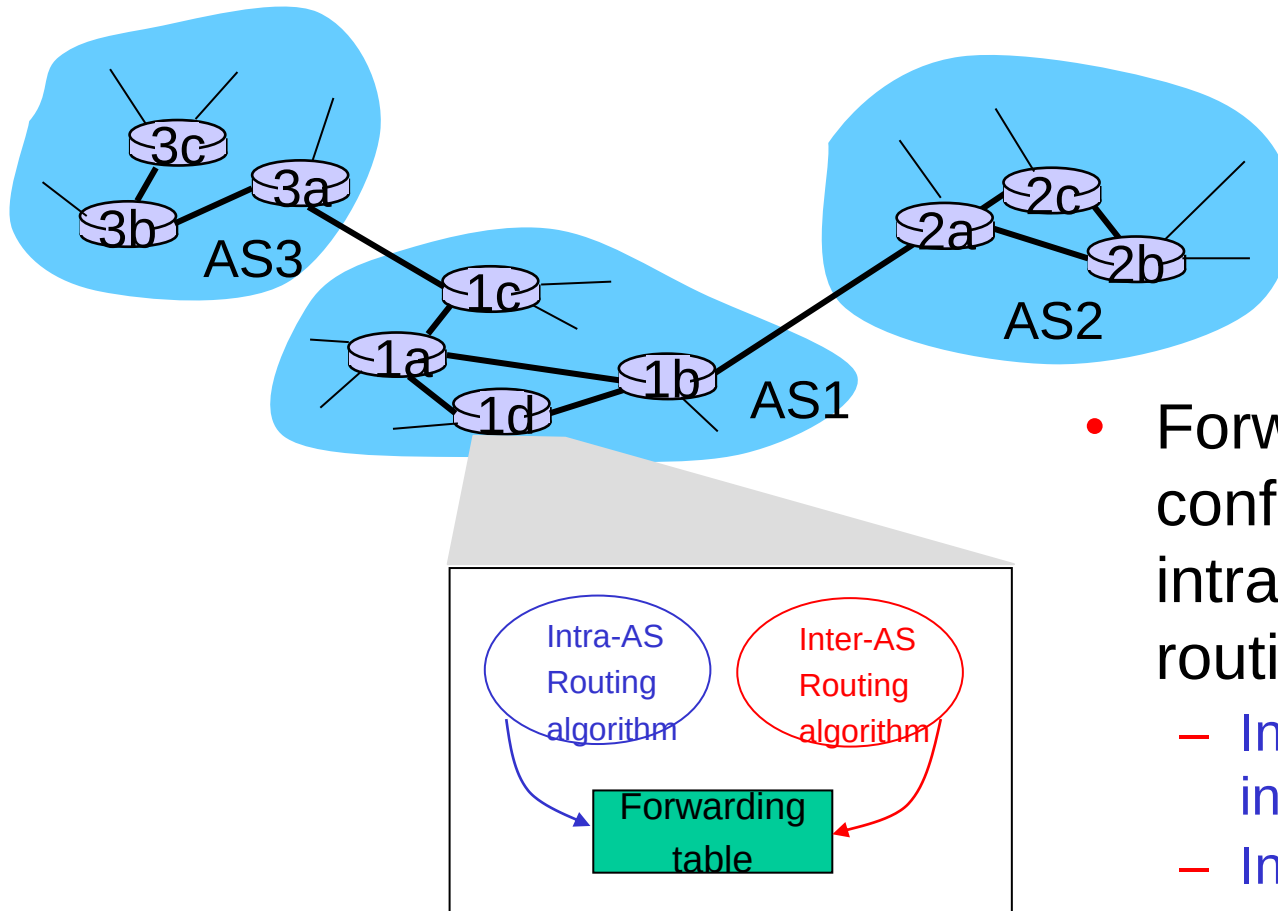
Hierarchical Routing

- aggregate routers into regions, “autonomous systems” (AS)
- routers in same AS run same routing protocol
 - “intra-AS” routing protocol
 - routers in different AS can run different intra-AS routing protocol

Gateway router

- Direct link to router in another AS

Interconnected ASes



- Forwarding table is configured by both intra- and inter-AS routing algorithm
 - Intra-AS sets entries for internal dests
 - Inter-AS & Intra-As sets entries for external dests

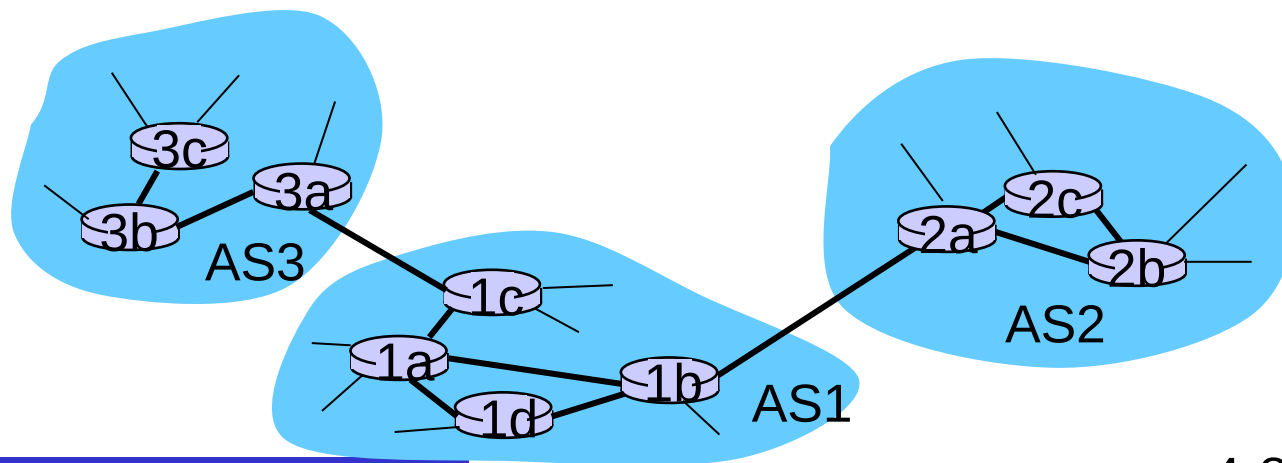
Inter-AS tasks

- Suppose router in AS1 receives datagram for which dest is outside of AS1
 - Router should forward packet towards one of the gateway routers, but which one?

AS1 needs:

1. to learn which dests are reachable through AS2 and which through AS3
2. to propagate this reachability info to all routers in AS1

Job of inter-AS routing!

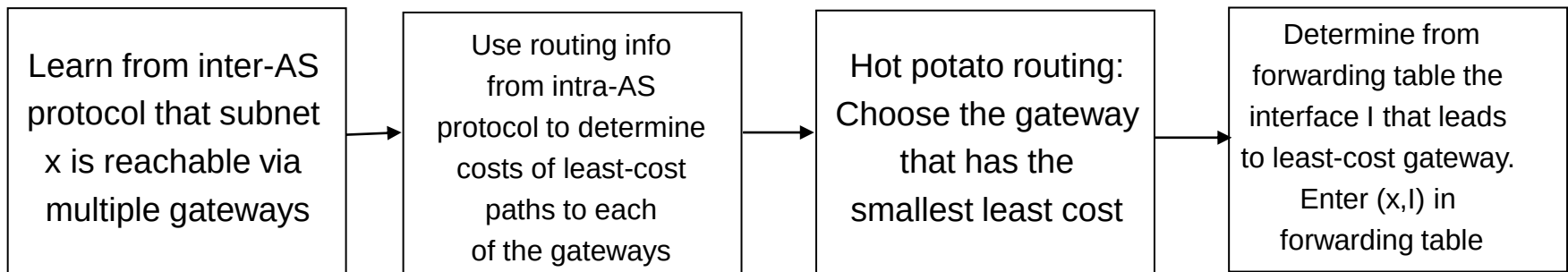


Example: Setting forwarding table in router 1d

- Suppose AS1 learns from the inter-AS protocol that subnet **x** is reachable from AS3 (gateway 1c) but not from AS2.
- Inter-AS protocol propagates reachability info to all internal routers.
- Router 1d determines from intra-AS routing info that its interface **/** is on the least cost path to 1c.
- Puts in forwarding table entry **(x, /)**.

Example: Choosing among multiple ASes

- Now suppose AS1 learns from the inter-AS protocol that subnet **x** is reachable from AS3 *and* from AS2.
- To configure forwarding table, router 1d must determine towards which gateway it should forward packets for dest **x**.
- This is also the job on inter-AS routing protocol!
- **Hot potato routing**: send packet towards closest of two routers.

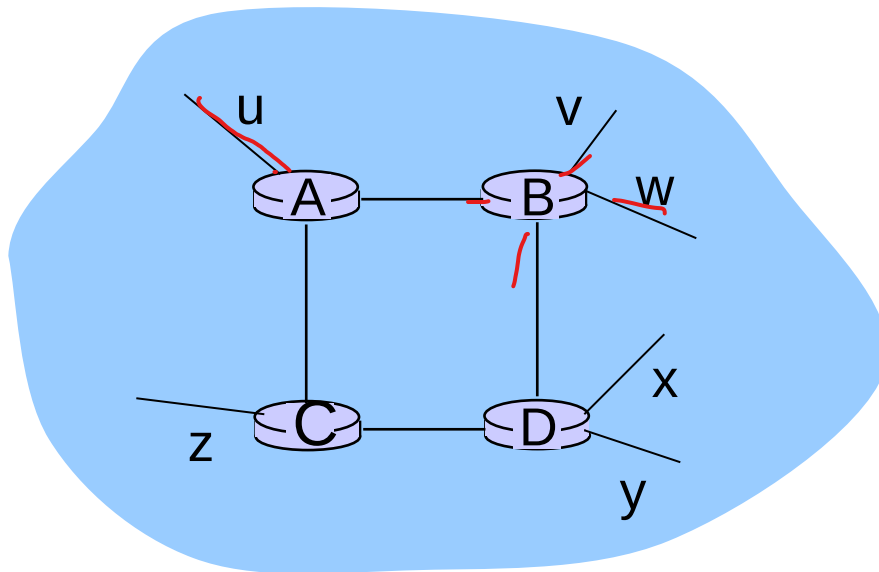


Intra-AS Routing

- Also known as **Interior Gateway Protocols (IGP)**
- Most common Intra-AS routing protocols:
 - RIP: Routing Information Protocol
 - OSPF: Open Shortest Path First
 - IGRP: Interior Gateway Routing Protocol (Cisco proprietary)

RIP (Routing Information Protocol)

- Distance vector algorithm
- Included in BSD-UNIX Distribution in 1982
- Distance metric: # of hops (max = 15 hops)

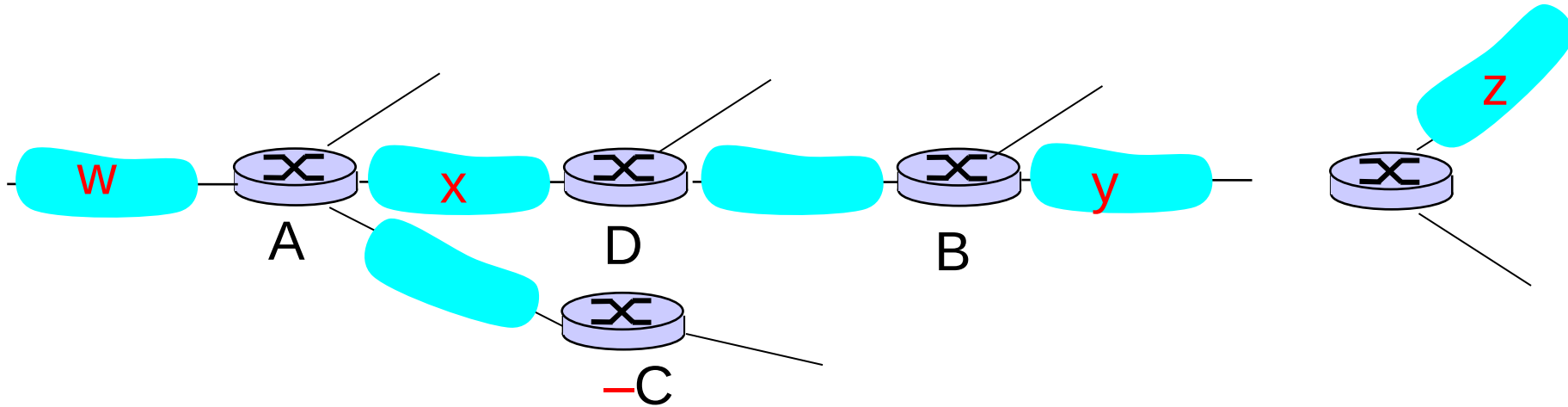


<u>destination</u>	<u>hops</u>
u	1
v	2
w	2
x	3
y	3
z	2

RIP advertisements

- Distance vectors: exchanged among neighbors every 30 sec via Response Message (also called **advertisement**)
- Each advertisement: list of up to 25 destination nets within AS

RIP: Example



Destination Network	Next Router	Num. of hops to dest.
W	A	2
y	B	2
z	B	7
x	--	1
....		

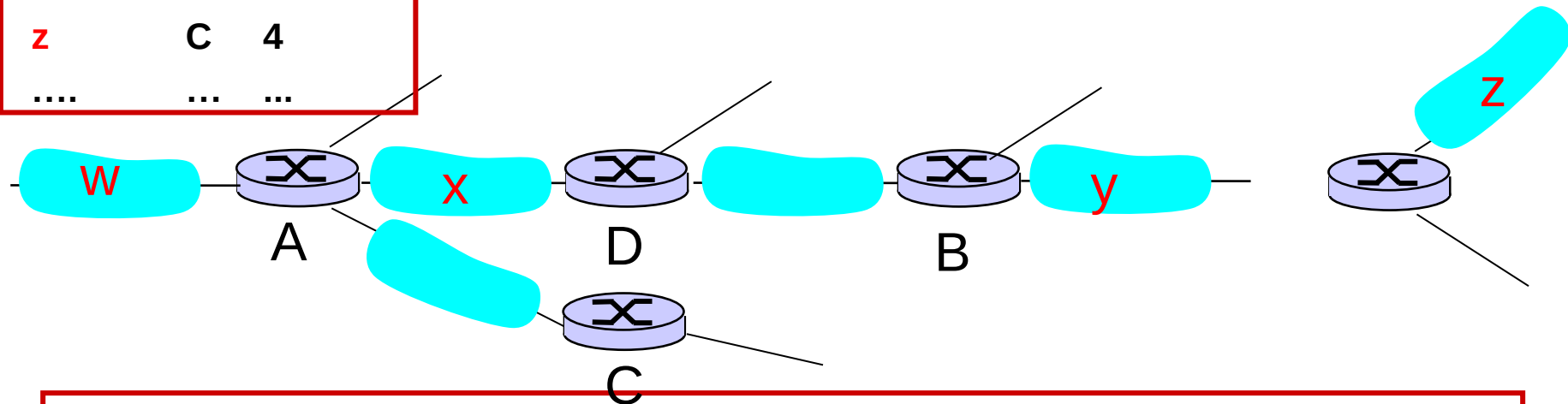
Routing table in D

RIP: Example

Dest Next hops

W - -
 X - -
 Z C 4

Advertisement
from A to D



Destination Network Next Router Num. of hops to dest.

W A 2
 Y B 2
 Z ~~B~~ A ~~7~~ 5
 X -- 1

Routing table in D

RIP: Link Failure and Recovery

If no advertisement heard after 180 sec --> neighbor/link declared dead

- routes via neighbor invalidated
- new advertisements sent to neighbors
- neighbors in turn send out new advertisements (if tables changed)
- link failure info quickly propagates to entire net
- poison reverse used to prevent ping-pong loops (infinite distance = 16 hops)

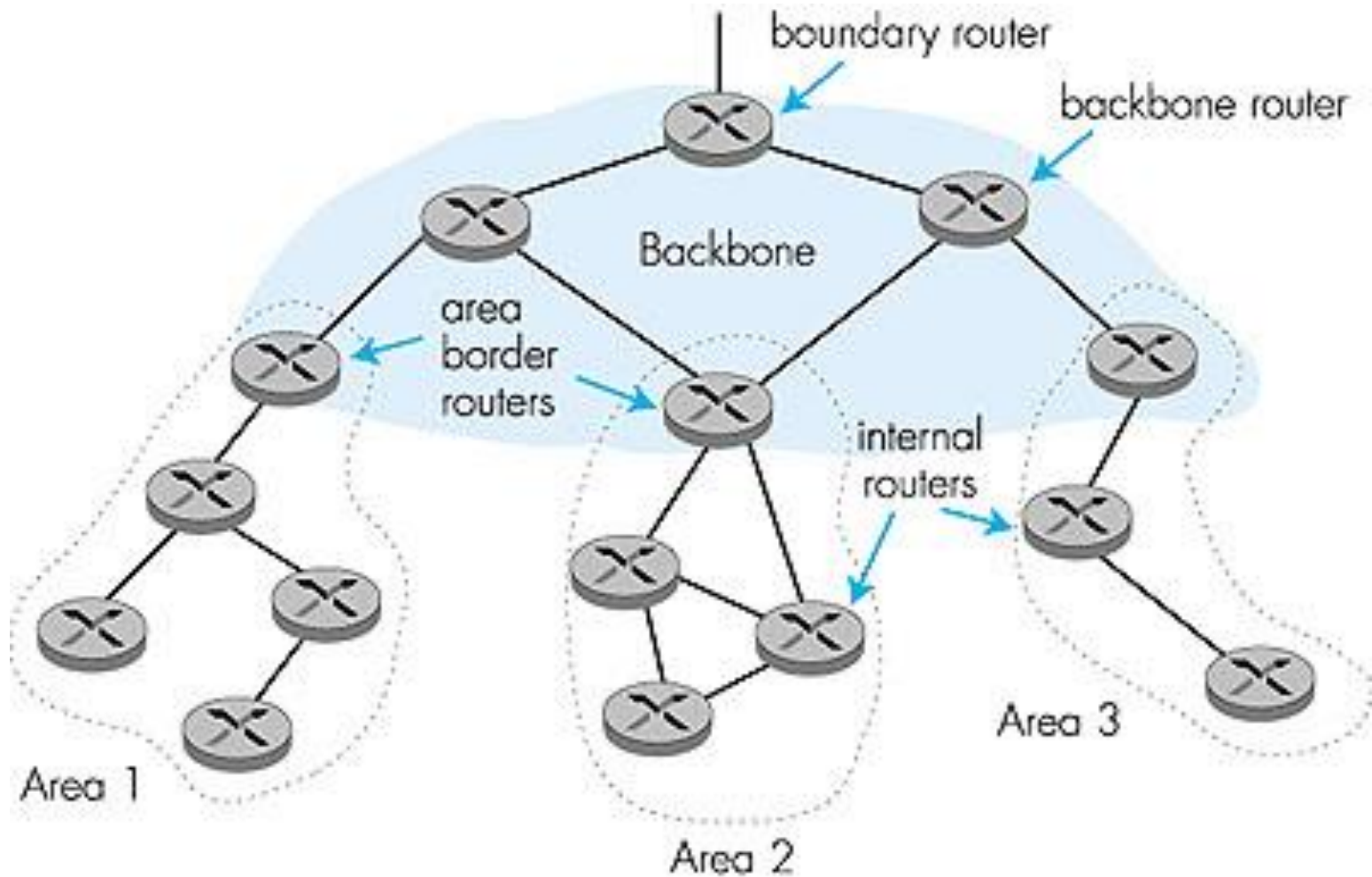
OSPF (Open Shortest Path First)

- “open”: publicly available
- Uses Link State algorithm
 - LS packet dissemination
 - Topology map at each node
 - Route computation using Dijkstra’s algorithm
- OSPF advertisement carries one entry per neighbor router
- Advertisements disseminated to **entire** AS (via flooding)
 - Carried in OSPF messages directly over IP (rather than TCP or UDP)

OSPF “advanced” features (not in RIP)

- **Security**: all OSPF messages authenticated (to prevent malicious intrusion)
- **Multiple** same-cost **paths** allowed (only one path in RIP)
- For each link, multiple cost metrics for different **TOS** (e.g., satellite link cost set “low” for best effort; high for real time)
- Integrated uni- and **multicast** support:
 - Multicast OSPF (MOSPF) uses same topology data base as OSPF
- **Hierarchical** OSPF in large domains.

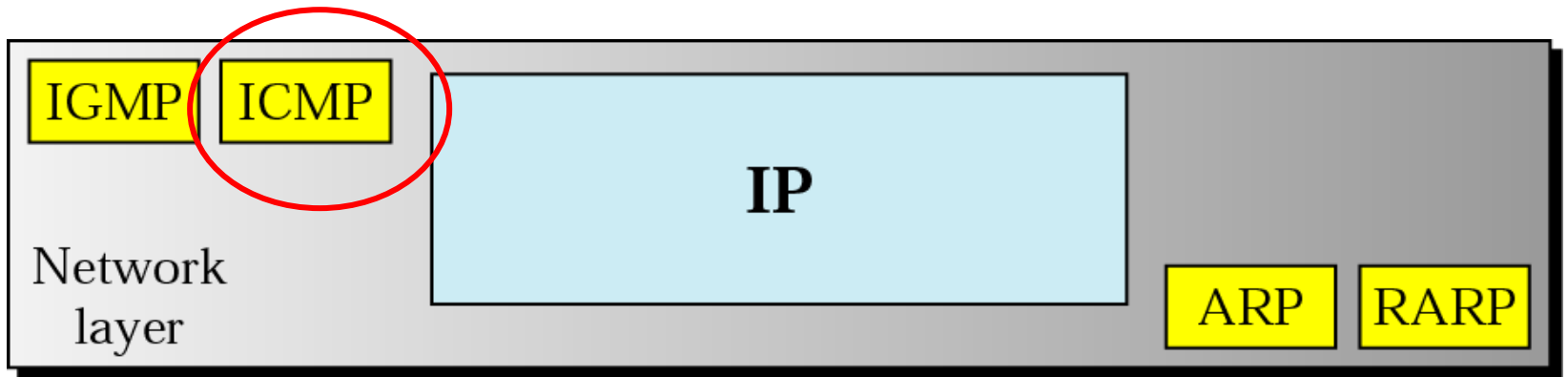
Hierarchical OSPF



Hierarchical OSPF

- **Two-level hierarchy:** local area, backbone.
 - Link-state advertisements only in area
 - each nodes has detailed area topology; only know direction (shortest path) to nets in other areas.
- **Area border routers:** “summarize” distances to nets in own area, advertise to other Area Border routers.
- **Backbone routers:** run OSPF routing limited to backbone.
- **Boundary routers:** connect to other AS's.

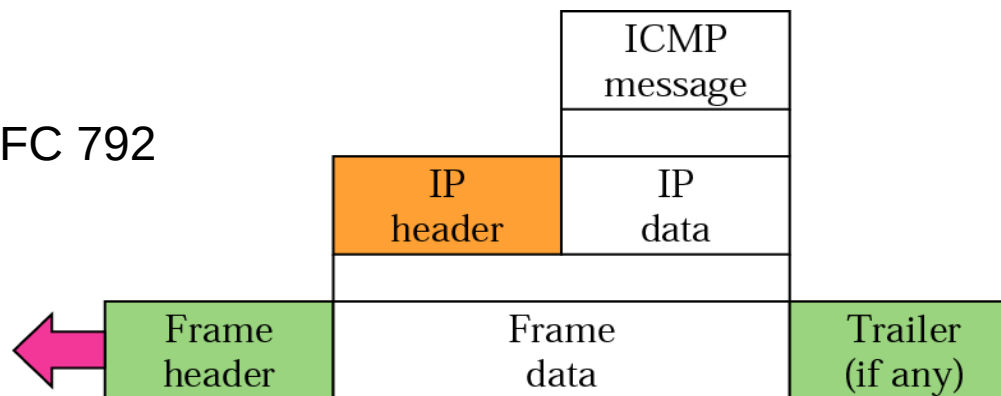
Network layer protocols



Internet Control Message Protocol

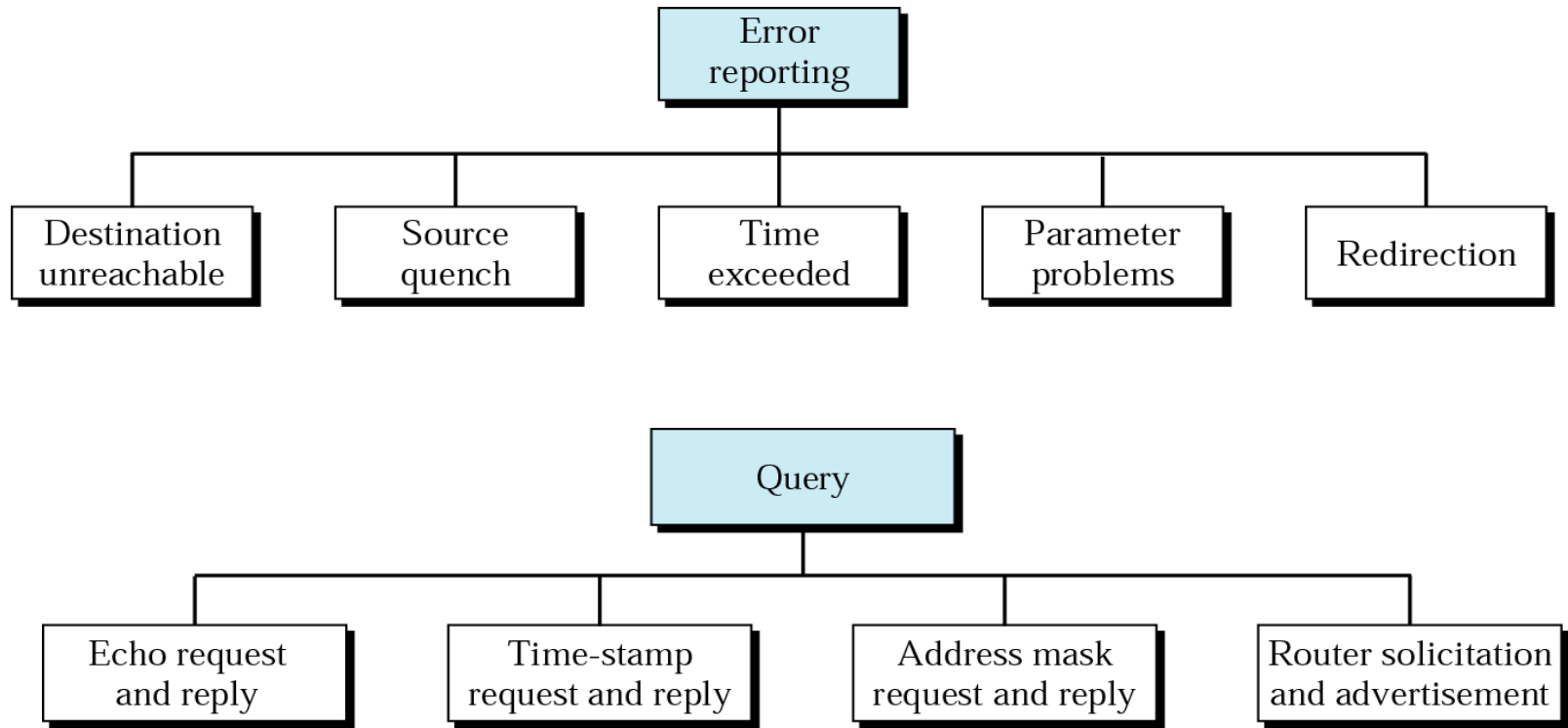
- What happens if the router must discard a datagram because he cannot find the final destination?
- What if **Time To Live** has a zero level?
- What if fragments of the datagram must be discarded because not all of them are received within a certain time?
- IP has no built-in mechanism to notify the host about these errors.
- Determining whether a host or a router is alive
- ICMP messages are first encapsulated into the IP packet
 - **error control**

specified in RFC 792

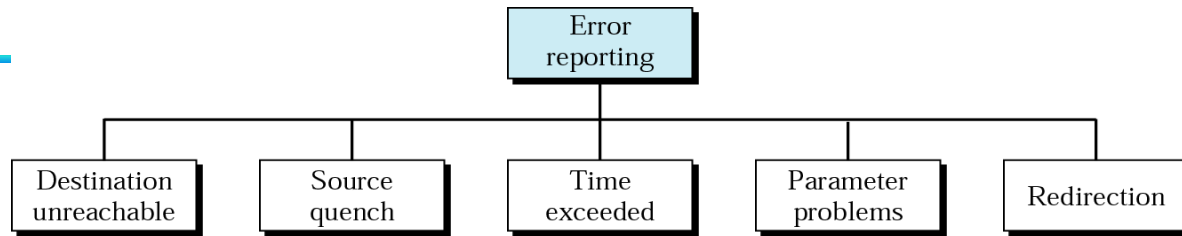


ICMP

-Types of messages-

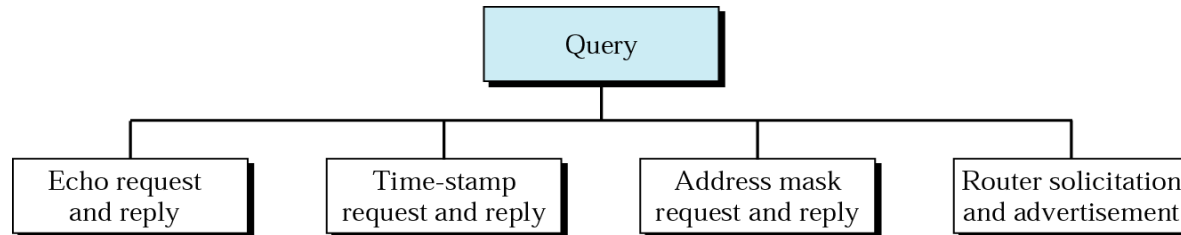


Error reporting



- **DU:** when a router cannot route a datagram or a host cannot deliver a datagram
- **SQ:** to add kind of flow & congestion control to IP
 - quench – to slow down
 - no communication among source hosts, routers and dest. hosts
- **TE:**
 - router generated when TTL=0
 - destination host generated when fragments not received within a certain time limit
- **PP:** if any value is missing in the datagram field
- **Redirection:** hosts do not take part in a routing process. Why? Their routing tables are not regularly updated. Default router performs the routing and sends its IP to a host.

Query



Diagnosis of network problems

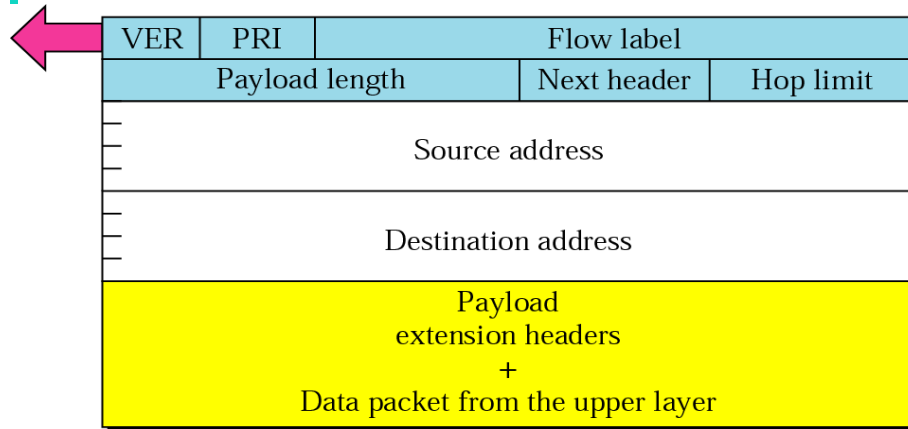
1. these messages determine whether 2 systems can communicate with each other
2. for 2 machines to determine the round-trip time for an IP datagram or clock synchronization
3. when host not aware of its netid, subnet, hostid it sends a AM request & the router responds with a AM reply
4. when a host wants to know the addresses of the routers connected to it

IPv6

Why was it introduced?

- IPv4 address space use was inefficient
- made to support real-time audio and video transmission
- to introduce security mechanism
 - encryption and authentication of data
- allow for extensions

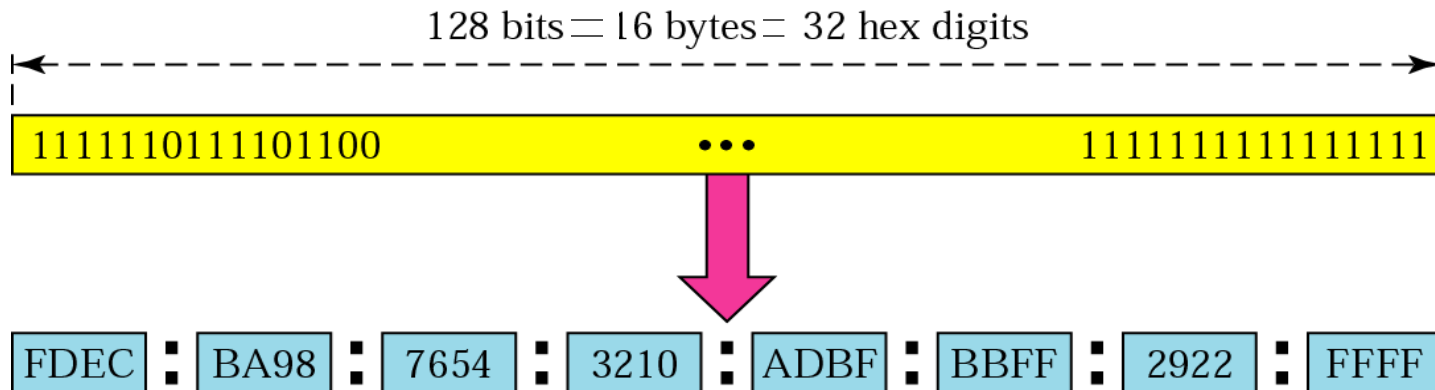
IPv6



- larger address space (128 bits)
- better header format
 - options separated from the base header
 - speeds up router processing
- allowance for extension
- support for resource allocation
 - flow label
 - to support audio & video
- support more security

IPv6 addresses

- hexadecimal colon notation



IPv6 - abbreviated addresses

Unabbreviated

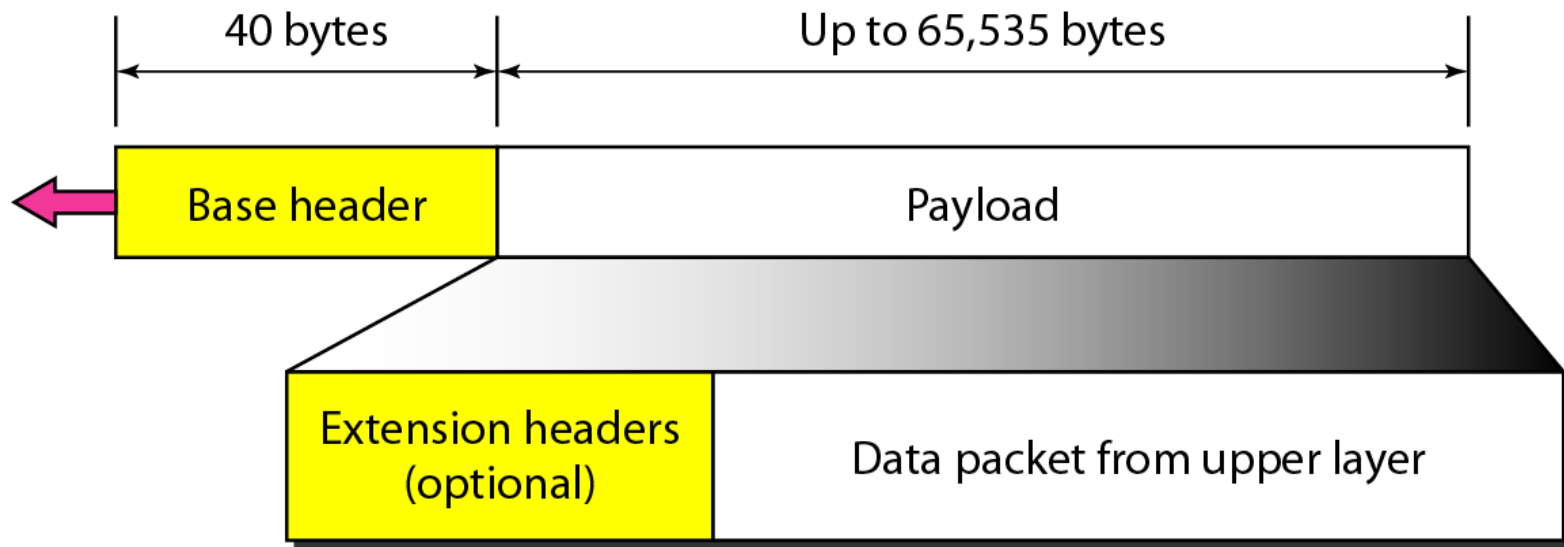
FDEC ■ BA98 ■ 0074 ■ 3210 ■ 000F ■ BBFF ■ 0000 ■ FFFF



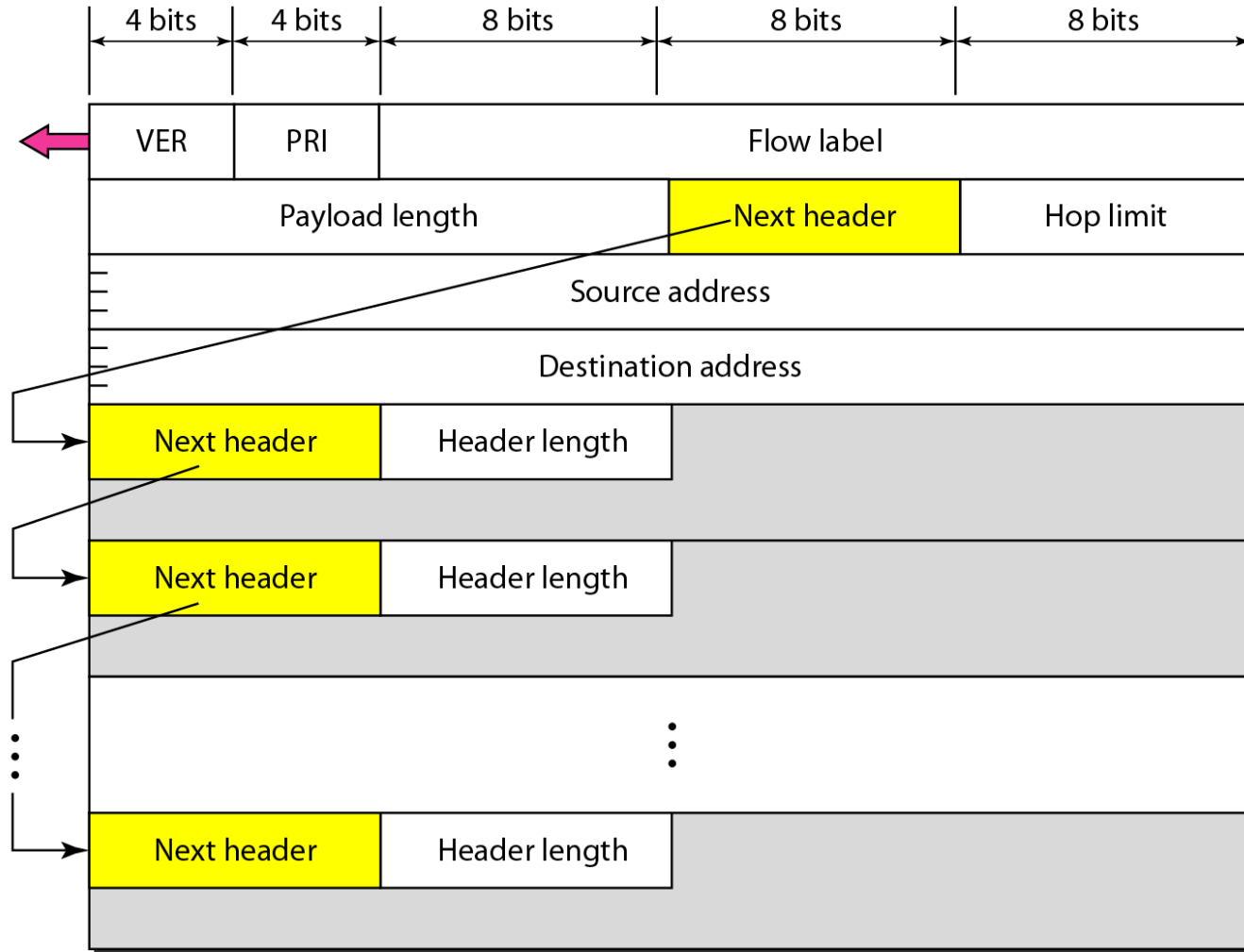
FDEC ■ BA98 ■ 74 ■ 3210 ■ F ■ BBFF ■ 0 ■ FFFF

Abbreviated

IPv6 datagram header and payload



Format of an IPv6 datagram



Next header codes for IPv6

<i>Code</i>	<i>Next Header</i>
0	Hop-by-hop option
2	ICMP
6	TCP
17	UDP
43	Source routing
44	Fragmentation
50	Encrypted security payload
51	Authentication
59	Null (no next header)
60	Destination option

Priorities for congestion-controlled traffic

<i>Priority</i>	<i>Meaning</i>
0	No specific traffic
1	Background data
2	Unattended data traffic
3	Reserved
4	Attended bulk data traffic
5	Reserved
6	Interactive traffic
7	Control traffic

Table 20.8 *Priorities for noncongestion-controlled traffic*

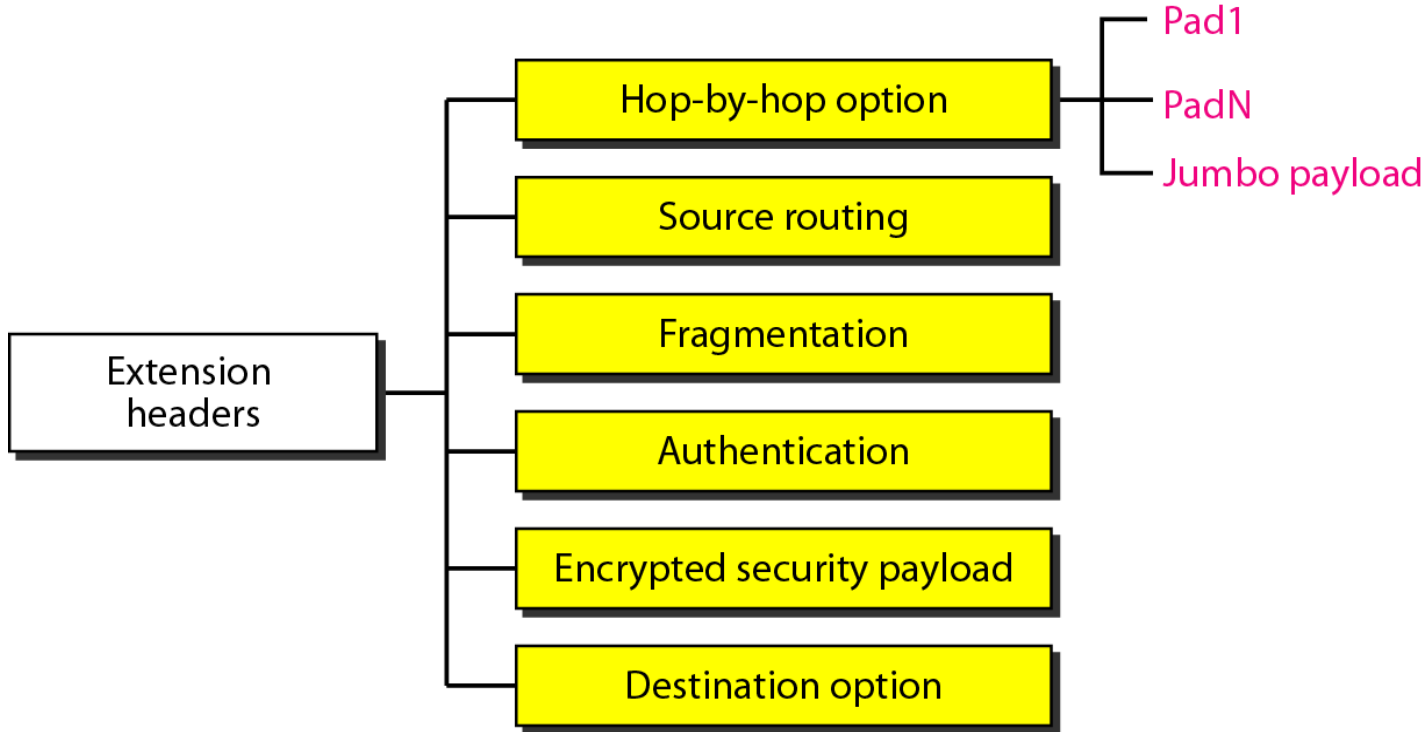
<i>Priority</i>	<i>Meaning</i>
8	Data with greatest redundancy
...	...
15	Data with least redundancy

Comparison between IPv4 and IPv6 packet headers

Comparison

1. The header length field is eliminated in IPv6 because the length of the header is fixed in this version.
2. The service type field is eliminated in IPv6. The priority and flow label fields together take over the function of the service type field.
3. The total length field is eliminated in IPv6 and replaced by the payload length field.
4. The identification, flag, and offset fields are eliminated from the base header in IPv6. They are included in the fragmentation extension header.
5. The TTL field is called hop limit in IPv6.
6. The protocol field is replaced by the next header field.
7. The header checksum is eliminated because the checksum is provided by upper-layer protocols; it is therefore not needed at this level.
8. The option fields in IPv4 are implemented as extension headers in IPv6.

Extension header types



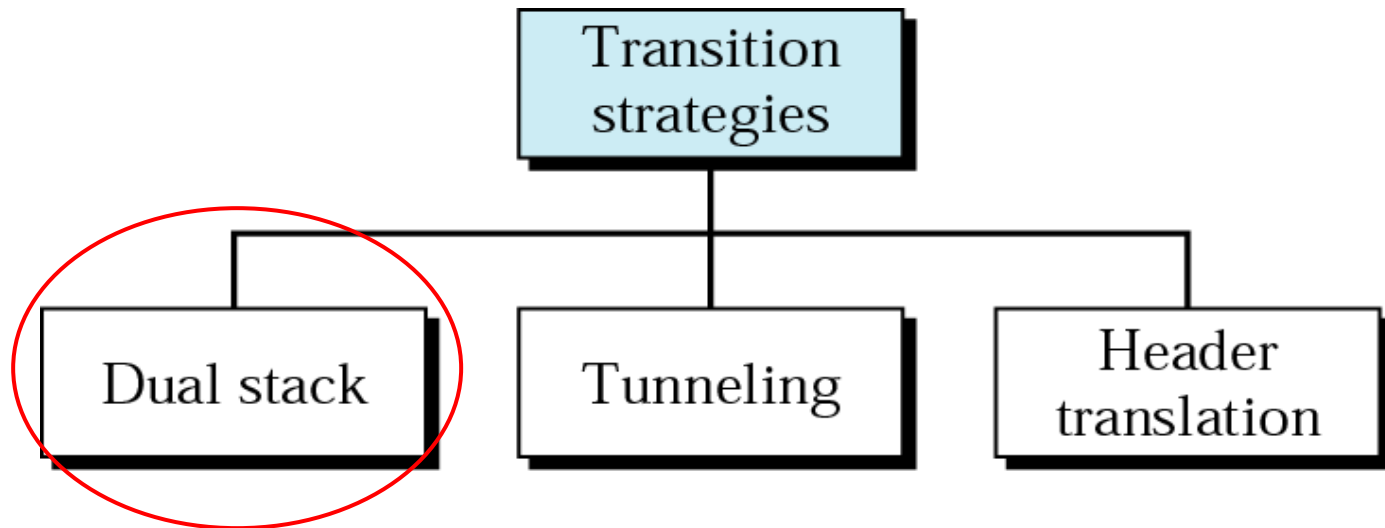
Comparison between IPv4 options and IPv6 extension headers

<i>Comparison</i>
1. The no-operation and end-of-option options in IPv4 are replaced by Pad1 and PadN options in IPv6.
2. The record route option is not implemented in IPv6 because it was not used.
3. The timestamp option is not implemented because it was not used.
4. The source route option is called the source route extension header in IPv6.
5. The fragmentation fields in the base header section of IPv4 have moved to the fragmentation extension header in IPv6.
6. The authentication extension header is new in IPv6.
7. The encrypted security payload extension header is new in IPv6.

IPv6 - fragmentation

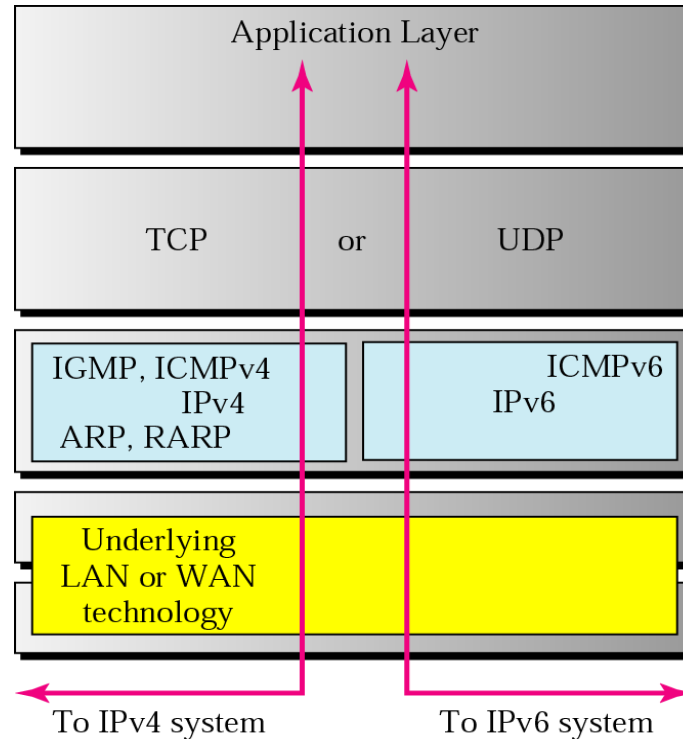
- Only the original host can fragment packets
 - note: in IPv4 both the hosts and the routers were required to fragment if the datagram $>$ MTU
- What is the advantage of this?

Transition from IPv4 to IPv6



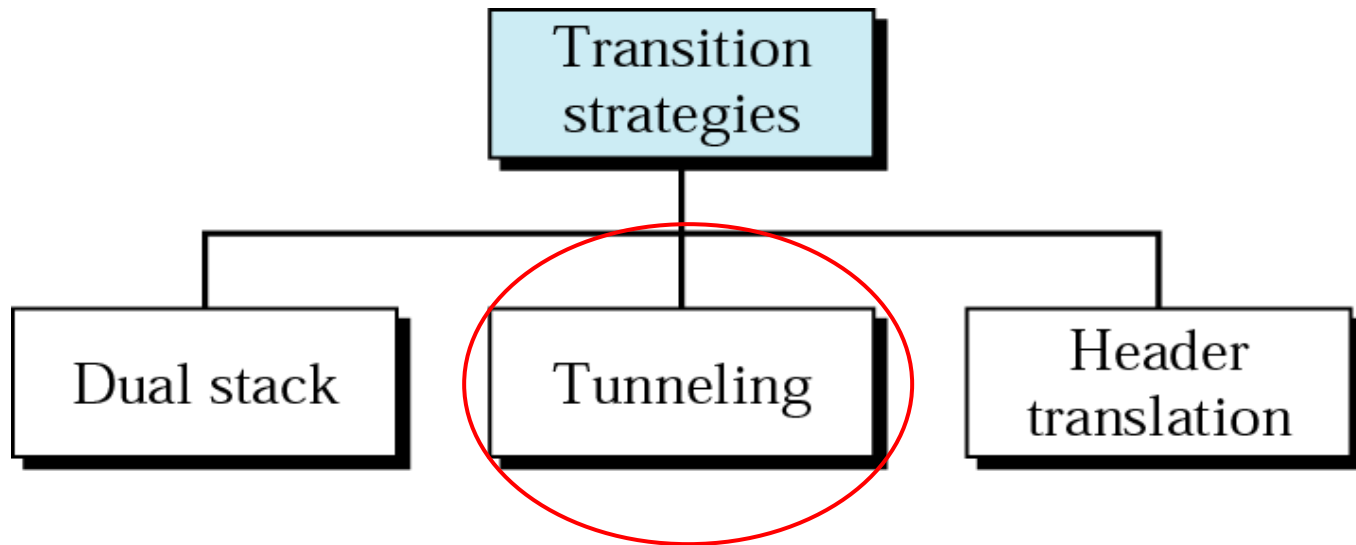
Dual stack

- station should run IPv4 & IPv6 simultaneously



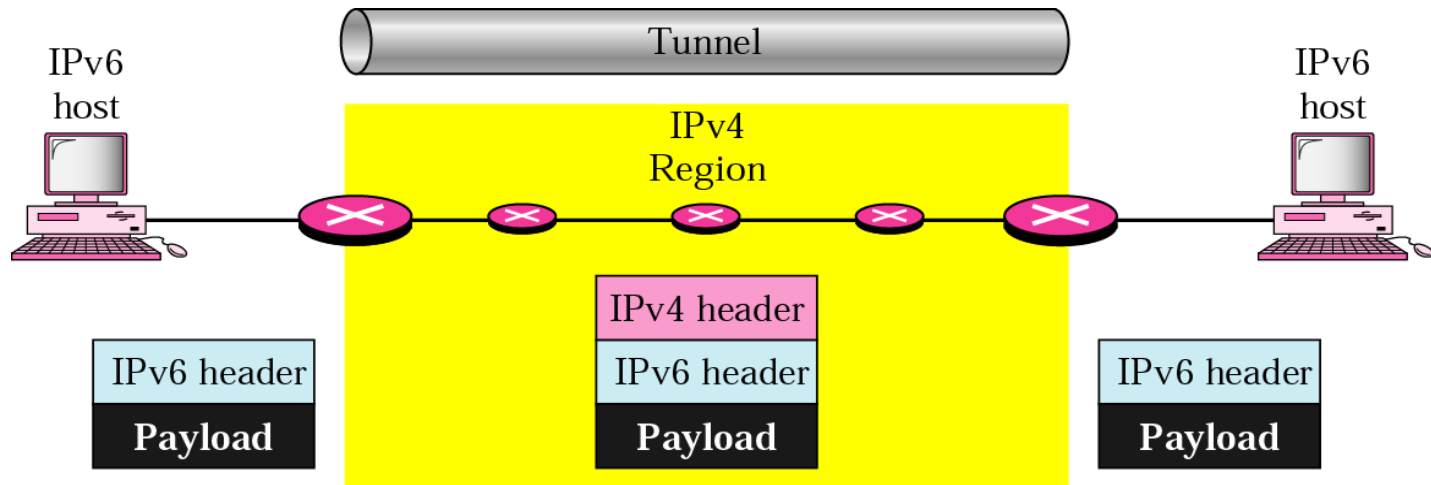
- to determine which version of a packet to use a source host queries the DNS

Transition from IPv4 to IPv6

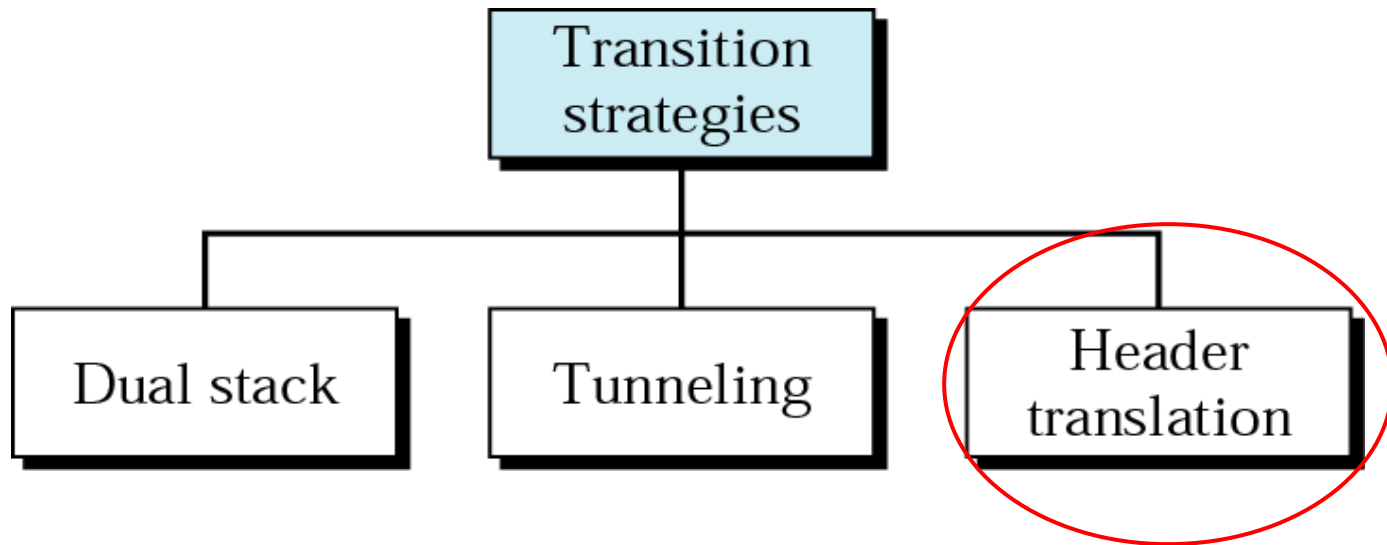


Tunneling

- IPv6 to IPv6 via IPv4

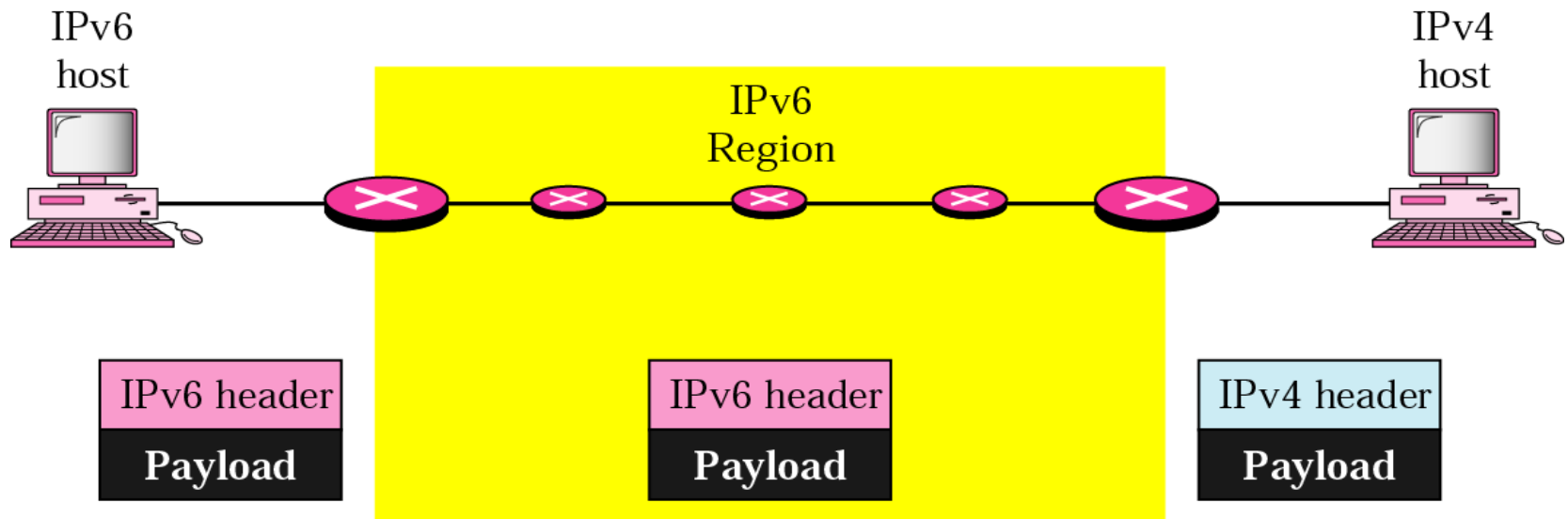


Transition from IPv4 to IPv6



Header translation

- When majority of the Internet hosts has moved to IPv6
- Tunneling cannot be implemented. Why?



Header translation

Header Translation Procedure

1. The IPv6 mapped address is changed to an IPv4 address by extracting the rightmost 32 bits.
2. The value of the IPv6 priority field is discarded.
3. The type of service field in IPv4 is set to zero.
4. The checksum for IPv4 is calculated and inserted in the corresponding field.
5. The IPv6 flow label is ignored.
6. Compatible extension headers are converted to options and inserted in the IPv4 header. Some may have to be dropped.
7. The length of IPv4 header is calculated and inserted into the corresponding field.
8. The total length of the IPv4 packet is calculated and inserted in the corresponding field.